

44. Bundeswettbewerb Informatik - 2. Runde - Aufgabe A2: Gießroboter

Teilnahme-ID: 79280

Bearbeiter/-innen dieser Aufgabe:

ChecksMc

8. Oktober 2026

Inhaltsverzeichnis

1	Glossar	3
2	Aufgabenstellung und Problemanalyse	4
2.1	Finale Annahmen	4
2.2	Garantie und Grenzen	4
3	Mathematische Modellierung	5
4	Lösungsansatz im Überblick	6
5	Initialisierung des Lösungsraums	8
5.1	Hilbert-Kurve	8
5.1.1	Greedy-Ansatz	9
5.1.2	Wahl des Ansatzes	10
6	Optimierungsverfahren	11
6.1	Genetischer Algorithmus (Optimierung 1)	11
6.1.1	Grundlagen	11
6.1.2	Fitnessberechnung	12
6.1.3	Population	13
6.1.4	Parameter	14
6.1.5	Wahl der Heuristik	15
6.1.6	Implementierung	15
6.2	Simulierte Abkühlung (Optimierung 2)	17
6.2.1	Grundlagen	17
6.2.2	Energieberechnung	18
6.2.3	Zustandsveränderung (Nachbarschaftsoperatoren)	19
6.2.4	Delta Berechnung	19
6.2.5	Abkühlung	20
6.2.6	Parameter	20
6.2.7	Wahl der Heuristik	21
6.2.8	Implementierung	22
6.3	Finale Parametrisierung	23
7	Umsetzung des Programms	24
7.1	Bibliotheken	25
7.2	Dateien	25
7.2.1	config.py	25
7.2.2	initialize.py	26
7.2.3	general.py	26
7.2.4	Hilbert.py	27
7.2.5	Greedy.py	27
7.2.6	GA_Improver.py	27
7.2.7	SA_Improver.py	28
7.2.8	validitycheck.py	29

7.2.9 main.py	29
8 Verwendung von KI	31
9 Beispiele	32
10 Vergleich der Lösungsansätze	49
10.1 Einleitung zum Vergleich	49
10.2 Vergleich nach Effizienz	49
10.3 Leistung	50
11 Fazit & Ausblick	52
12 Vergleich mit KI-Numba Ansatz	53
12.1 Konkrete Implementierungsunterschiede	53
12.2 Beispielrun roboter03 (SA vs. GA)	53
12.3 Einordnung des KI-Numba-Ansatzes	54
13 Literaturverzeichnis	55
Bibliografie	55
14 Quellcode	56
14.1 Hauptablauf	56
14.2 Einlesen + Distanzmatrix	56
14.3 Initiaillösung: Hilbert + Greedy	56
14.4 Simulated Annealing (SA)	57
14.5 Genetischer Algorithmus (GA)	57
14.6 Validierung + Ausgabe	57
14.7 Ergänzende Module	58

1 Glossar

Roboter: Ausführende Einheit für die Bewässerungsaufgabe; jeder Route ist genau ein Roboter zugeordnet.

Route: Fahrt eines Roboters, die dem Aufbau Startpunkt (Ladestation) $\rightarrow$ Baum $\rightarrow \dots \rightarrow$ Startpunkt (Ladestation) folgt. Die Länge der Route darf den Maximalwert $L_{\max}$ nicht überschreiten.

Ladestation (Depot): Start- und Endpunkt einer Route; standardmäßig wird der erste Knoten der Route als Position der Ladestation festgelegt.

MDEVRP: Multiple Depot Electric Vehicle Routing Problem, die formelle Klassifikation des Problems, bei der Routen unter Berücksichtigung von Distanzbeschränkungen optimiert werden.

Metaheuristik: Stochastisches Optimierungsverfahren, das auch für NP-schwere Probleme in polynomieller Zeit qualitativ hochwertige Näherungslösungen generiert

Energiefunktion: Bewertungsfunktion bei der Simulierten Abkühlung, die die Qualität einer Lösung quantifiziert; je niedriger der Wert, desto besser die Lösung.

Fitnessfunktion: Bewertungsfunktion beim Genetischen Algorithmus, analog zur Energiefunktion die Qualität eines Chromosoms (Lösung) bewertet.

Hamiltonkreis: Ein geschlossener Weg, der jeden Knoten genau einmal besucht und zum Startknoten zurückkehrt

Vektorisierung (Numpy): Ein Verfahren der parallelen Datenverarbeitung, bei dem Operationen auf ganze Arrays statt auf einzelne Elemente angewendet werden, was die Berechnung einer Distanzmatrix massiv beschleunigt.

Inkrementelle Energieberechnung: Eine Optimierungstechnik im SA, bei der nach einer Zustandsänderung nur die betroffenen Routen neu bewertet werden.

SA: Ein stochastisches Suchverfahren zur Approximation globaler Optima, das den physikalischen Abkühlungsprozess von Metallen imitiert, um lokale Minima zu überwinden.

GA: Ein evolutionärer Optimierungsalgorithmus, der Prinzipien der natürlichen Selektion (Mutation, Crossover und Selektion) nutzt, um über Generationen hinweg die besten Lösungen für ein Problem zu finden.

2 Aufgabenstellung und Problemanalyse

Es wird ein Programm gefordert, das auf Basis der Baumkoordinaten und einer maximal zulässigen Fahrstrecke die kleinste Anzahl von Gießrobotern bestimmt, die benötigt wird, um alle Bäume zu gießen. Jede Route startet und endet an einer Ladestation.

Dafür werden zwei Ziele definiert:

- Anzahl der Roboter minimieren (von der Aufgabe gefordertes Ziel)
- Gesamtroutenlänge nur als schwach gewichtetes Nebenziel berücksichtigen (relevant erst nach Roboter Menge und Validität)

Dabei bleiben zwei Punkte in der Aufgabenstellung offen:

- Wie viele Ladestationen dürfen eingesetzt werden?
- Gibt es für jeden Roboter eine eigene Ladestation, oder ist es möglich, dass eine Ladestation mehreren Robotern zugeordnet ist?

Anhand des dritten BWINF-Beispiels (roboter03) lässt sich erkennen, dass die Anzahl der Ladestationen größer als eins sein muss, da es keine Möglichkeit gibt, alle Bäume von einer Ladestation aus zu erreichen, ohne die maximale Fahrstrecke zu überschreiten.

Da die Anzahl der Ladestationen nicht eingeschränkt wird, wird in der Bearbeitung angenommen, dass jede Route eine eigene Ladestation hat. Diese Modellierung vereinfacht die Optimierung, ohne die Aufgabenlogik zu verändern.

2.1 Finale Annahmen

Für die restliche Bearbeitung gelten die folgenden Modellannahmen:

- Jede Route erhält genau eine eigene Ladestation.
- Jede Route wird als geschlossener Ring betrachtet, also mit gleichem Start- und Endpunkt.
- Hauptziel ist die Minimierung der Roboteranzahl.
- Die Gesamtfahrstrecke fließt nur als schwach gewichteter Qualitätsindikator ein.

Diese Annahmen spezifizieren die durch die Aufgabenstellung offen gelassenen Punkte und bilden die Grundlage für die mathematische Modellierung und die Lösungsansätze.

2.2 Garantie und Grenzen

Die Implementierung garantiert eine vollständig validierte Lösung mit korrekten Routen, Einhaltung des Streckenlimits und konsistenter Ausgabe. Es wird jedoch bewusst keine globale Optimalitätsgarantie im mathematischen Sinn gegeben, da die eingesetzten Metaheuristiken (SA und GA) als heuristische Verbesserungsverfahren konzipiert sind, die in polynomieller Zeit qualitativ hochwertige Näherungslösungen liefern, aber nicht notwendigerweise das absolute Optimum erreichen.

Für die finale Auswahl gilt eine harte Priorisierung der Roboteranzahl: Eine metaheuristisch gefundene Lösung wird nur dann übernommen, wenn sie valide ist und nicht mehr Roboter als die Initiallösung benötigt. Die Gesamtdistanz ist in SA und GA als kleiner Gewichtungsterm enthalten und dient daher nur als weiches Tie-Breaking innerhalb dieses Primärziels.

3 Mathematische Modellierung

Definition des Problems:

Das vorliegende Problem lässt sich als Variante eines Multiple-Depot- Routing-Problems auffassen und ist damit NP-schwer, da bereits der Basisfall mit nur einer Route und einer festen Ladestation das Traveling-Salesman-Problem als Teilproblem enthält, das nachweislich NP-schwer ist [1]. Die zusätzlichen Randbedingungen erhöhen die Komplexität weiter, da sie die Anzahl der möglichen Lösungen drastisch erhöhen und die Suche nach einer optimalen Lösung erschweren.

Da die Aufgabenstellung keine feste Obergrenze für die Anzahl der Ladestationen vorgibt, wird, wie oben bereits beschrieben, angenommen, dass jede Route eine eigene Ladestation hat. Dies führt zu einem Modell, bei dem die Anzahl der Routen (Roboter) direkt mit der Anzahl der Ladestationen korreliert, was die Optimierung vereinfacht, ohne die Logik der Aufgabe zu verändern.

Zusätzliche Randbedingungen

Im Vergleich zum klassischen VRP kommen folgende Einschränkungen hinzu:

- Die Anzahl und Position der Depots (Ladestationen) sind nicht vordefiniert.
- Das primäre Optimierungsziel ist die Minimierung der Anzahl der Routen (Roboter), nicht der Gesamtdistanz.
- Keine Route darf die maximale Fahrstrecke $L_{\max}$ überschreiten.

Aufgrund dieser Randbedingungen wächst die Anzahl möglicher Lösungen mit steigender Baumanzahl schnell an. Für größere Instanzen sind daher heuristische oder metaheuristische Verfahren praktikabler als eine vollständige Enumeration [1].

Mathematische Definition:

Dabei wird definiert:

Jeder Baum wird genau einmal in genau einer Route besucht.

Jede Route bildet einen Ring, Start und Ende liegen an derselben Ladestation.

Kein Roboter überschreitet die maximale Fahrstrecke $L_{\max}$.

Alle Ladestationen sind frei wählbar; es werden keine Positionen vorgegeben.

und

k beschreibt die Anzahl der Roboter.

$l(r)$ die Länge einer Route r .

$L_{\max}$ die maximale Fahrstrecke.

N beschreibt die Anzahl an Bäumen.

Gegeben seien:

- Eine Menge an Bäumen $T = \{t_1, t_2, \dots, t_N\}$ mit $t_i = (x_i, y_i)$ als kartesischen Koordinaten des Baumes t_i .
- Die maximal zulässige Fahrstrecke $L_{\max} > 0$.
- Der euklidische Abstand zwischen zwei Punkten P_1 und P_2 : $d(P_1, P_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.
- Eine Distanzmatrix $D \in \mathbb{R}^{N \times N}$ mit $D(i, j) = d(t_i, t_j)$ für alle $i, j \in \{1, \dots, N\}$.

Die Baummenge T wird dann in k disjunkte, nicht-leere Teilmengen $r_1, r_2, \dots, r_k$ aufgeteilt. Zusätzlich wird für jede Route eine Ladestation benötigt.

Es bestehen dabei weitere Bedingungen:

- Jeder Baum muss genau einer Teilmenge zugeordnet sein.
- Die Teilmengen $r_1, r_2, \dots, r_k$ sind disjunkt.
- Keine Teilmenge darf leer sein.
- Jede Route muss insgesamt kürzer als oder gleich $L_{\max}$ sein.

Wenn eine Route als Folge $r = (t_1, t_2, \dots, t_m)$ beschrieben wird, dann gilt für ihre Länge:

$$l(r) = d(D, t_1) + \sum_{i=1}^{m-1} d(t_i, t_{i+1}) + d(t_m, D) \leq L_{\max}$$

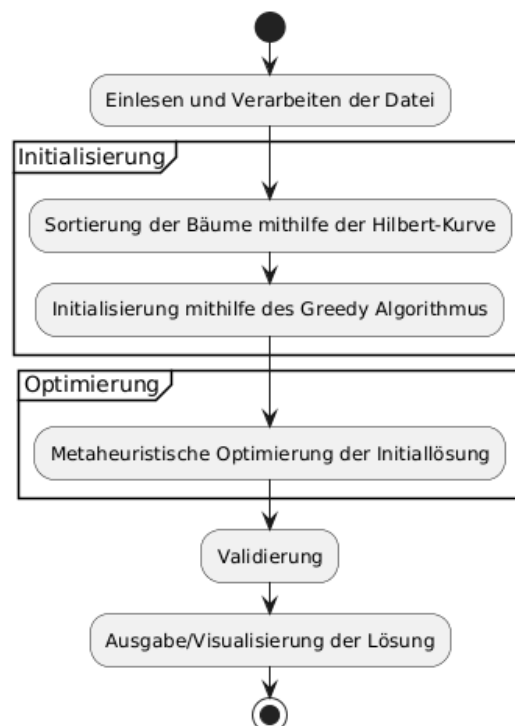
wobei D das Depot beziehungsweise die Ladestation beschreibt.

Das Hauptziel ist die Minimierung der Anzahl der Roboter k ; als Nebenziel wird zusätzlich die Gesamtfahrstrecke minimiert.

Da die Ladestationen frei wählbar sind und jeder Baum nur einmal besucht wird, kann die Ladestation auf dem jeweiligen Ring als Start- und Endpunkt gewählt werden. In der praktischen Darstellung wird dafür der erste Baum der Route als Referenzpunkt verwendet.

4 Lösungsansatz im Überblick

Dieses Kapitel bietet einen Überblick über den angewendeten Lösungsansatz



Für die Lösungsfindung wird ein zweistufiges Konzept verwendet mit der Initialisierung, also der Berechnung einer Startlösung und der metaheuristischen Optimierung. Diese zweistufige Vorgehensweise erhöht die Laufzeiteffizienz drastisch, da Metaheuristiken komplexer sind und fördert außerdem die globale Explorationsfähigkeit und das anschließende Ergebnis.

Das Diagramm bildet den Arbeitsablauf des Programms ab. Es besteht aus sieben „Phasen“:

1. Einlesen und Verarbeiten der Datei

Die Daten werden aus der Datei gelesen und in die maximale Länge $L_{\max}$ und die Bäume verarbeitet.

2. Initialisierungsphase

- Sortierung der Bäume mithilfe der Hilbert-Kurve

Die zweidimensionalen Koordinaten der Bäume werden mithilfe der Hilbert-Kurve auf eine eindimensionale Reihenfolge abgebildet, die die räumliche Lokalität beibehält.

- Initialisierung mithilfe des Greedy-Algorithmus

Basierend auf der Hilbert-Sortierung wird ein kapazitätsbasierter Greedy-Algorithmus ausgeführt. Er weist die Bäume nacheinander der aktuellen Route zu. Falls die Grenze überschritten wird, wird der Baum in die nächste Route gesteckt

3. Optimierungsphase (eingerahmter Funktionsblock)

In dieser Phase wird die Initiallösung durch eine metaheuristische Optimierung verbessert, um die Anzahl der Roboter zu minimieren und die Gesamtfahrstrecke zu optimieren. Hierfür wurden zwei Verfahren implementiert: die Simulierte Abkühlung (Simulated Annealing, SA) und der Genetische Algorithmus (GA). Beide Verfahren erkunden den Lösungsraum, um lokale Optima zu verlassen und eine möglichst hochwertige Näherungslösung zu finden, die das globale Optimum in vielen Fällen gut annähern kann.

4. Validierung

Die erzeugte Lösung wird auf die Einhaltung aller Aufgabenbedingungen geprüft. Es wird kontrolliert, ob:

- jede Route die maximale Fahrstrecke nicht überschreitet,
- jeder Baum genau einmal besucht wird
- keine leeren Routen enthalten sind.

5. Ausgabe/Visualisierung der Lösung

Die Lösung wird zum einen als maschinenlesbare Textdatei mit der Anzahl der Roboter und der genauen Reihenfolge der Bäume pro Route, zum anderen optional als Graph mit den Routen im Koordinatensystem um die Ergebnisse nachvollziehbarer darzustellen.

5 Initialisierung des Lösungsraums

Die Initialisierung legt den Grundstein für die Effizienz des nachfolgenden Optimierungsschritts. Ziel ist es, eine qualitativ hochwertige Initiallösung zu generieren, die bereits eine grundlegende logische Struktur aufweist, ohne dabei die Laufzeitkomplexität signifikant zu erhöhen.

5.1 Hilbert-Kurve

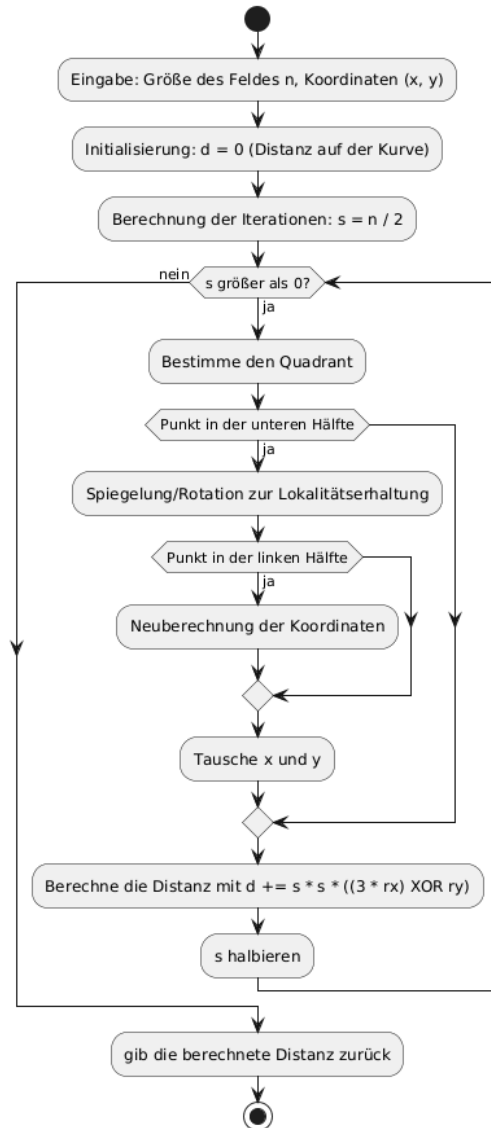


Abbildung 2: Berechnung der Hilbert-Distanz

Die Hilbert-Kurve ist eine fraktale raumfüllende Kurve, die mithilfe von rekursiver Iteration konstruiert wird. In diesem Kontext dient sie als mathematischer Clustering-Ansatz, um eine zweidimensionale Ebene auf eine eindimensionale Linie abzubilden.

Der Kern des Verfahrens ist die sogenannte xy2d-Logik (xy to distance), dabei wird jedem Punkt (x, y) im zweidimensionalen Gitter ein eindeutiger Skalarwert d zugeordnet, der die Position des Punktes entlang der Hilbert-Kurve beschreibt.

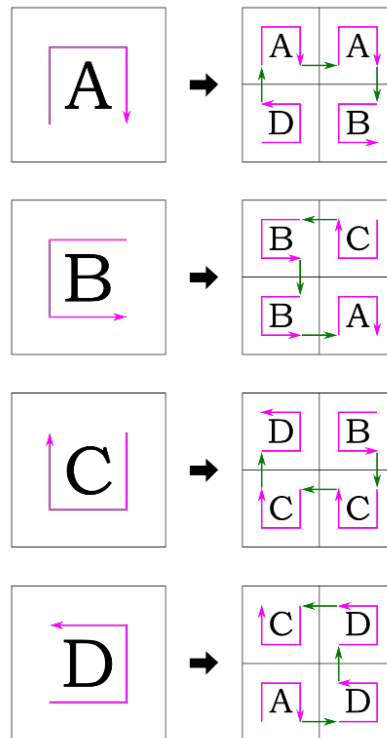
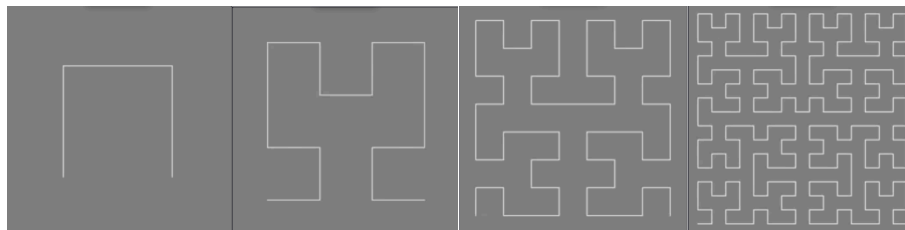


Abbildung 3: xy2d Logik [2]


Abbildung 4: Beispiel einer Hilbertkurve mit der Größe $n=4$ [3]

Der Vorteil der Hilbert-Kurve gegenüber einfacher Sortierung ist ihre Lokalitätserhaltung. Punkte, die auf der zweidimensionalen Ebene nah beieinander liegen, behalten mit hoher Wahrscheinlichkeit auch in der eindimensionalen Hilbertkurve eine geringe Distanz zueinander bei.

5.1.1 Greedy-Ansatz

Basierend auf der durch die Hilbert-Kurve definierten Reihenfolge wird ein kapazitätsbasierter ($L_{\max}$) Greedy-Algorithmus zur Erstellung der Initiaillösung eingesetzt.

Der Algorithmus initialisiert eine leere Route und weist so lange Bäume in der Reihenfolge einer gegebenen Liste (in diesem Fall die nach dem Hilbert-Index Sortierten Bäume) der Route zu, bis die Kapazität (d. h. die Strecke inklusive Rückweg $L_{\max}$ überschreiten würde) erreicht wird. Falls dies der Fall ist, wird eine neue Route gestartet. Der Prozess wird so lange wiederholt bis alle Bäume zugewiesen wurden, die dadurch entstandenen Routen werden dann als Initiaillösung verwendet.

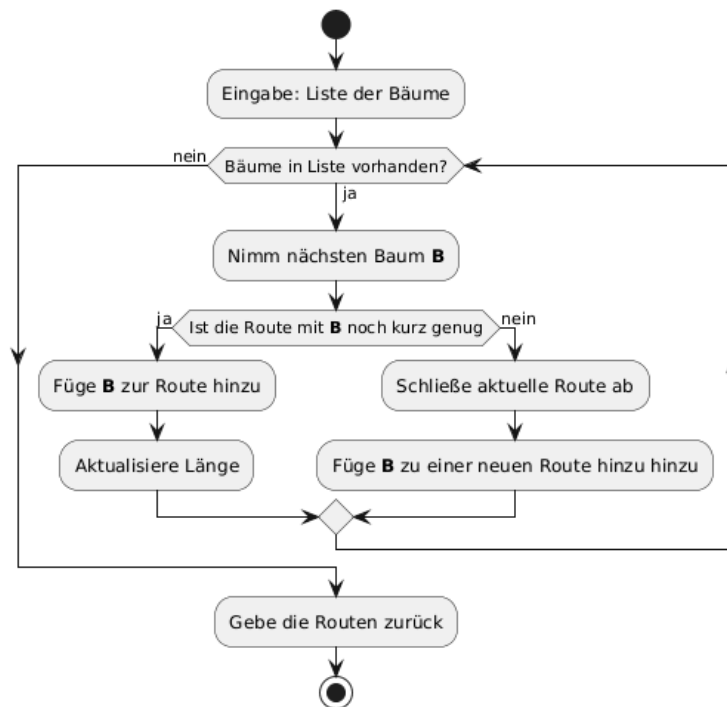


Abbildung 5: Ablauf des Greedy-Algorithmus

5.1.2 Wahl des Ansatzes

Für die räumliche Sortierung der Baumkoordinaten wurde die Hilbert-Kurve gewählt. Diese Wahl basiert auf drei Vorteilen gegenüber alternativen Sortier- und Clusteringverfahren.

1. Erhaltung der räumlichen Lokalität

Die Hilbert-Kurve ist eine fraktale, raumfüllende Kurve, die zweidimensionale Koordinaten auf eine eindimensionale Sequenz abbildet. Punkte, die auf der kartesischen Ebene nah beieinander liegen, behalten auch in der eindimensionalen Sortierung mit hoher Wahrscheinlichkeit eine geringe Distanz zueinander.

2. Geringe Laufzeitkomplexität und hohe Skalierbarkeit

Die Berechnung der Hilbert-Sortierung weist eine Laufzeitkomplexität von $O(N \log N)$ auf. Sie ist damit effizienter als iterative Clusteringverfahren wie k-Means oder DBSCAN.

3. Reduzierung der vorzeitigen Konvergenz in metaheuristischen Verfahren

Ein großes Problem bei metaheuristischen Optimierungsverfahren ist die vorzeitige Konvergenz in einem lokalen Optimum. Eine qualitativ hochwertige Initiallösung mindert dieses Risiko maßgeblich.

6 Optimierungsverfahren

Im folgenden Kapitel werden die verwendeten metaheuristischen Verfahren genauer umschrieben.

Das vorliegende Problem des Gießroboters lässt sich als NP-schweres MDEVRP klassifizieren. Exakte Lösungsverfahren sind daher selbst bei geringen Knotenzahlen nicht praxistauglich. Deshalb kommen zwei metaheuristische Optimierungsverfahren zum Einsatz, die auf der Initiallösung aufbauen: die Simulierte Abkühlung (Simulated Annealing, SA) und der Genetische Algorithmus (Genetic Algorithm, GA).

Beide Verfahren verfolgen das übergeordnete, zweistufige Optimierungsziel der Aufgabenstellung:

1. Primäre Ziele: Minimierung der Anzahl der benötigten Roboter (Routen)
Valides Ergebnis erzeugen: das Ergebnis muss valide sein
2. Sekundäres Ziel: Die Gesamtfahrstrecke wird nur schwach gewichtet berücksichtigt, um bei sonst vergleichbaren Lösungen unnötige Umwege zu reduzieren.

Die Simulierte Abkühlung fokussiert dabei auf die iterative Optimierung einer einzelnen Lösung, während der Genetische Algorithmus über eine Population von Lösungen den gesamten Lösungsraum explorierend untersucht. Beide Ansätze können in begrenzter Laufzeit qualitativ hochwertige, praxistaugliche Näherungslösungen erzeugen, die jedoch nicht unbedingt optimal sind.

Für Reproduzierbarkeit und Vergleichbarkeit der Ergebnisse ist es wichtig, die stochastischen Elemente der Verfahren zu kontrollieren. Dafür werden feste Seed-Listen verwendet, die es ermöglichen, mehrere Starts reproduzierbar zu gestalten und die Ergebnisse verschiedener Instanzen fair zu vergleichen. Die verwendeten Seeds sind zufällig generierte Zahlen, die in Tests gut abgeschnitten haben, und sind in der Abgabe unter `config.py` enthalten.

Bei der Verwendung von Genetischen Algorithmen (GA) oder Simulated Annealing (SA) ist die Konsistenz der Zufallszahlen entscheidend. Es wird dringend davon abgeraten, sich bei der Nutzung von Pythons `multiprocessing`-Modul auf einen einfachen, globalen Seed innerhalb der Worker-Prozesse zu verlassen, da dies zu inkonsistenten Ergebnissen führen kann. Stattdessen sollten für jeden Prozess individuelle Seeds generiert und übergeben werden, um sicherzustellen, dass die Zufallszahlen in jedem Prozess unabhängig und reproduzierbar sind. Dies kann durch die Verwendung von `numpy.random.SeedSequence` oder durch manuelles Generieren von Seeds vor der Erstellung der Prozesse erreicht werden.

6.1 Genetischer Algorithmus (Optimierung 1)

6.1.1 Grundlagen

Der genetische Algorithmus ist ein populationsbasiertes heuristisches Optimierungsverfahren, das Prinzipien der biologischen Evolution simuliert, mithilfe von Selektion, Rekombination und Mutation. Anstatt iterativ einen einzelnen Zustand zu verbessern (wie bei der simulierten Abkühlung), wird gleichzeitig eine Menge von Kandidatenlösungen betrachtet, wodurch eine breitere Exploration des Suchraums ermöglicht wird und die Gefahr der vorzeitigen Konvergenz reduziert wird [4].

Jede Lösung wird in diesem Kontext als Chromosom dargestellt, das aus einer linearen Permutation der Baum-IDs besteht. Um das Chromosom in eine gültige Routenlösung umzuwandeln, wird ein Greedy-Decodierungsverfahren verwendet, das Ähnlichkeiten mit dem in der Initialisierungsphase aufweist. Diese Kodierung hat zwei entscheidende Vorteile:

- Sie vereinfacht die Implementierung der genetischen Operatoren, da alle gängigen Kreuzungs- und Mutationsoperatoren für Permutationen direkt angewendet werden können [5].

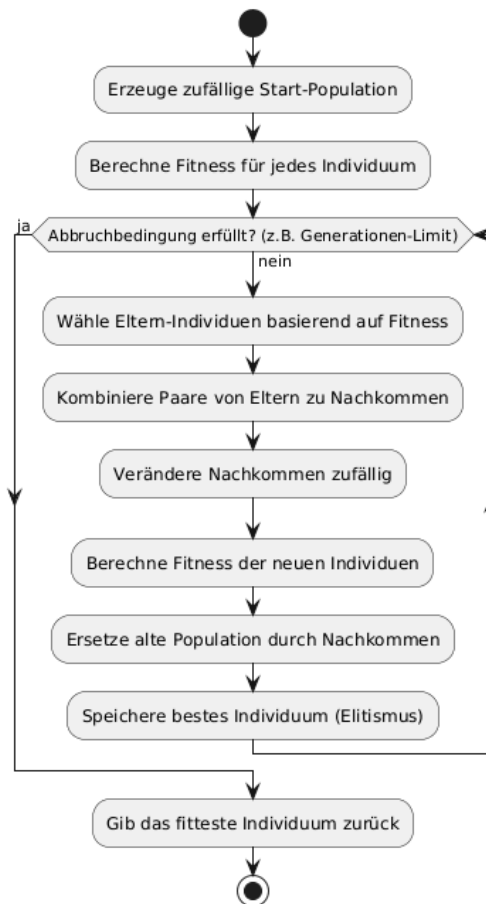


Abbildung 6: Ablauf eines GA

Sie garantiert, dass nach jeder Operation alle Bäume genau einmal im Chromosom enthalten sind. Dadurch entstehen keine ungültigen Lösungen.

6.1.2 Fitnessberechnung

Die Fitnessfunktion bewertet die Qualität eines Chromosoms und ist eng an die Energiefunktion der Simulierten Abkühlung angepasst. Auch hier gilt: Je niedriger der Fitnesswert, desto besser ist die Lösung. Die Funktion lautet wie folgt:

$$F(C) = \alpha \cdot k + \beta \cdot \sum_{r \in S} I(l(r) > L_{\max} + m) \cdot \left(100 + 900 \cdot \frac{g}{\max}(1, G)\right) \cdot (l(r) - L_{\max})^2 + \gamma \cdot \sum_{r \in S} l(r) \quad (1)$$

Dabei beschreiben:

- C das bewertete Chromosom,
- k die Anzahl der Routen (Roboter) nach der Dekodierung,
- g die aktuelle Generation,
- G die Gesamtanzahl der Generationen,
- $m = \text{GA_OVERLENGTH_MARGIN} \cdot L_{\max}$ die erlaubte Überlängen-Marge,
- $\left(100 + 900 \cdot \frac{g}{\max}(1, G)\right)$ ist ein Fortschrittsfaktor, der die Strafe für überlange Routen mit zunehmender Generation erhöht, um gegen Ende des Optimierungsprozesses nur noch valide Lösungen zu akzeptieren.
- α, β, γ die Gewichtungparameter.

Gegenüber der Energiefunktion der Simulierten Abkühlung gibt es zwei Anpassungen an die Besonderheiten des GA:

- Die Überlänge wird quadratisch erhöht.
- Im Optimierungsprozess erlaubt der Decoder eine Überschreitung der maximalen Fahrstrecke um eine Marge ($GA_OVERLENGTH_MARGIN = 0.05$). Dies verhindert, dass der Algorithmus bereits in frühen Generationen auf eine starre Routenstruktur festgelegt wird, und erhält die Vielfalt der Population. Bei der Generierung der finalen Lösung wird diese Marge deaktiviert, sodass nur vollständig valide Routen ausgegeben werden.

6.1.3 Population

Initialpopulation

Die Startpopulation wird aus zwei Komponenten zusammengesetzt, um eine optimale Balance zwischen Lösungsqualität und genetischer Vielfalt zu erreichen:

- Ein Chromosom entspricht der hochwertigen Initiallösung aus der Hilbert-Sortierung und dem Greedy-Algorithmus. Das stellt mit dem Elitismus sicher, dass die Population immer eine qualitativ hochwertigere Lösung hat.
- Alle weiteren Chromosomen der Population bestehen aus zufälligen Permutationen der Baum-IDs. Dies ermöglicht eine hohe Vielfalt in der Population und ermöglicht es einen größeren Lösungsraum zu betrachten

Selektion

Für die Auswahl der Elternindividuen wird ein Turnierselktionsverfahren mit einer Turniergröße von 3 eingesetzt. Dabei werden wiederholt drei zufällige Individuen aus der Population gezogen, und das Individuum mit dem niedrigsten Fitnesswert wird als Elternteil ausgewählt.

Dieses Verfahren hat gegenüber anderen Selektionsmethoden zwei entscheidende Vorteile:

1. Es erzeugt einen kontrollierten Selektionsdruck. Dadurch werden qualitativ hochwertige Individuen bevorzugt, ohne die genetische Vielfalt der Population einzuschränken
2. Es ist rechnerisch effizient und unabhängig von der Skalierung der Fitnesswerte, was die Stabilität des Algorithmus erhöht.

Rekombination (Crossover) [6]

Als Kreuzungsoperator wird der Ordered Crossover 1 (OX1) eingesetzt. Dieser Operator ist speziell für permutationsbasierte Chromosomen entwickelt und eignet sich optimal für das vorliegende Routenplanungsproblem, da er die Reihenfolgeinformationen der Elternchromosomen erhält und gleichzeitig gültige Nachkommen ohne doppelte oder fehlende Bäume generiert [5].

Mutation

Als Mutationsoperator wird eine Inversion mit einer Wahrscheinlichkeit von $GA_MUTATION_PROB=0.1$ pro Chromosom eingesetzt. Dabei wird ein zufällig ausgewählter Teilabschnitt des Chromosoms in seiner Reihenfolge umgedreht.

Diese Mutationsart hat zwei entscheidende Vorteile für das Problem:

- Sie führt nur lokale Veränderungen am Chromosom durch
- Sie erhält die Gültigkeit des Chromosoms, da keine Baum-IDs hinzugefügt, entfernt oder vertauscht werden

Die Mutation dient vor allem dazu, die genetische Vielfalt der Population zu erhalten und eine vorzeitige Konvergenz in einem lokalen Optimum zu vermeiden.

Lokale Optimierung (memetischer Algorithmus)

Mit einer Wahrscheinlichkeit von $GA_LOCAL_IMPROVE_PROB=0.15$ wird auf dem generierten Nachkommenchromosom eine lokale 2-Opt-Optimierung durchgeführt. Dabei werden in jeder Route Kantenpaare getauscht, um die Fahrstrecke zu reduzieren, bis keine weitere Verbesserung mehr möglich ist.

Diese Erweiterung macht den Algorithmus zu einem memetischen Verfahren. Die Anzahl der Iterationen der lokalen Optimierung steigt mit der Fortschreitung der Generationen, sodass auch hier am Ende des Prozesses der Fokus auf der Feinoptimierung der Lösung liegt.

Elitismus

Um sicherzustellen, dass die beste gefundene Lösung im Laufe des evolutionären Prozesses nicht verloren geht, wird ein Elitismus-Mechanismus implementiert: Die zwei besten Individuen der aktuellen Generation werden unverändert in die nächste Generation übernommen. Dies stabilisiert die Konvergenz des Algorithmus und garantiert, dass sich die Qualität der besten Lösung über die Generationen hinweg nicht verschlechtert.

6.1.4 Parameter

Auch hier wurden die Parameter nach intensiven acht-stündigen Tests berechnet. Hierfür wurden folgende Bereiche für die Variablen verwendet

```
ALPHA_VALUES = [100, 500, 1000]
BETA_VALUES = [200, 400, 600]
GAMMA_VALUES = [1, 3, 5]
MUT_PROBS = [0.01, 0.05]
POP_SIZES = [25, 50]
GEN_COUNTS = [25, 50]
LOCAL_IMPROVE_PROBS = [i/100 for i in range(0, 31, 15)]
STANDARD_POPULATIONS = [50, 100]
OVERLENGTH_MARGINS = [0.01, 0.05]
MAX_POPULATIONS = [200, 500]
MAX_GENERATIONS = [200, 500]
```

Es wurden aber nur 2 Durchläufe pro Datei gemacht, da es mehr Parameter gibt.

Als Testinstanzen dienten roboter04, roboter07 und roboter09.

Die besten Ergebnisse wurden mit den Parametern

```
GA_ALPHA = 2000
GA_BETA = 300
GA_GAMMA = 3
GA_MUTATION_PROB = 0.1
GA_POPULATION = 50
GA_GENERATIONS = 100
GA_STANDARD_POPULATION = 50
GA_LOCAL_IMPROVE_PROB = 0.3
```

erzielt.

In der Implementierung zeigte sich jedoch, dass diese Parameter oft nicht die besten Ergebnisse lieferten, da die optimale Parametrierung stark von der Problemgröße abhängt. Daher wurde eine größenbasierte Preset-Logik eingeführt, um die Parameter automatisch an die Problemgröße anzupassen.

Die Größenordnung der Gewichte zeigt die Zielpriorisierung deutlich: Die Anzahl der Routen dominiert die Bewertung, während die Distanz über $\gamma = 3$ nur als kleiner Korrekturterm eingeht.

Die Ergebnisse der Tests sind auch in der Abgabe zu finden. Der Code (GApam.py) zum Testen und die Analyse der Daten wurde mithilfe von KI gemacht, während die finale Auswertung und Interpretation eigenständig erfolgten.

Um die Robustheit gegenüber verschiedenen Problemgrößen sicherzustellen, nutzt der Genetische Algorithmus eine größenbasierte Preset-Logik.

- Small (N<333)
- Medium (N<1926)
- Large

mit folgenden Konfigurationen:

```
GA_CONFIG_SMALL = {
    "alpha": 100,
    "beta": 600,
    "gamma": 1,
```

```

    "mutation_prob": 0.05,
    "population": 50,
    "generations": 50,
    "local_improve_prob": 0.15,
    "standard_population": 50,
    "overlength_margin": 0.05,
    "max_population": 500,
    "max_generations": 500,
}

GA_CONFIG_MEDIUM = {
    "alpha": 500,
    "beta": 600,
    "gamma": 1,
    "mutation_prob": 0.01,
    "population": 50,
    "generations": 50,
    "local_improve_prob": 0.15,
    "standard_population": 50,
    "overlength_margin": 0.05,
    "max_population": 500,
    "max_generations": 500,
}

GA_CONFIG_LARGE = {
    "alpha": 500,
    "beta": 400,
    "gamma": 5,
    "mutation_prob": 0.05,
    "population": 50,
    "generations": 50,
    "local_improve_prob": 0.15,
    "standard_population": 50,
    "overlength_margin": 0.01,
    "max_population": 500,
    "max_generations": 500,
}

```

6.1.5 Wahl der Heuristik

Der Genetische Algorithmus wurde als Hauptverfahren implementiert, da er viele Vorteile hat, die speziell für das vorliegende Routenplanungsproblem relevant sind:

- Durch die populationsbasierte Arbeitsweise kann der Algorithmus gleichzeitig mehrere Regionen des Lösungsraums untersuchen. Dies reduziert das Risiko der vorzeitigen Konvergenz.
- Der Genetische Algorithmus liefert auch bei großen Problemgrößen in deutlich kürzerer Zeit gute Näherungslösungen als die Simulierte Abkühlung. Dies macht ihn zu einer praktikablen Alternative für sehr große Eingabedatensätze, bei denen die Simulierte Abkühlung mehr Zeit benötigt.
- Der Algorithmus kann leicht durch zusätzliche genetische Operatoren oder Lokalsuchverfahren erweitert werden, um die Lösungsqualität weiter zu steigern.
- Die Größe der Population und die Anzahl der Generationen können an die Problemgröße angepasst werden, sodass der Algorithmus sowohl bei kleinen als auch bei sehr großen Datensätzen in einer akzeptablen Zeit und konsistent gute Ergebnisse liefert.

6.1.6 Implementierung

Bei der Implementierung wurden Änderungen vorgenommen, um die Aufgabe besser zu lösen:

- Sowohl die Anzahl der Chromosomen pro Generation als auch die Gesamtanzahl der Generationen werden an die Anzahl der Bäume angepasst. Dies optimiert das Gleichgewicht zwischen Lösungsqualität und Laufzeit für jede Problemgröße.

- Die Decodierungsfunktion verfügt über zwei Betriebsmodi: Im Optimierungsmodus wird eine geringe Überlängen-Marge erlaubt, um die Vielfalt der Population zu erhalten. Im Ausgabemodus wird diese Marge deaktiviert, sodass nur vollständig valide Routen generiert werden, die alle Randbedingungen einhalten.
- Die Anzahl der Iterationen der 2-Opt-Lokalsuche steigt mit der Anzahl der vergangenen Generationen. Dadurch wird sichergestellt, dass in frühen Generationen die Vielfalt der Population erhalten bleibt, während in späten Generationen die Lösung fein optimiert wird.
- Nach den Ergebnissen des Parametertests fällt auf, dass $\alpha < \beta$ in vielen Fällen besser abschneidet.

Nach Abschluss des Optimierungsprozesses werden leere Routen automatisch entfernt und die finale Lösung auf die Einhaltung der maximalen Fahrstrecke validiert. Damit wird sichergestellt, dass die ausgegebene Lösung die implementierten Bedingungen erfüllt.

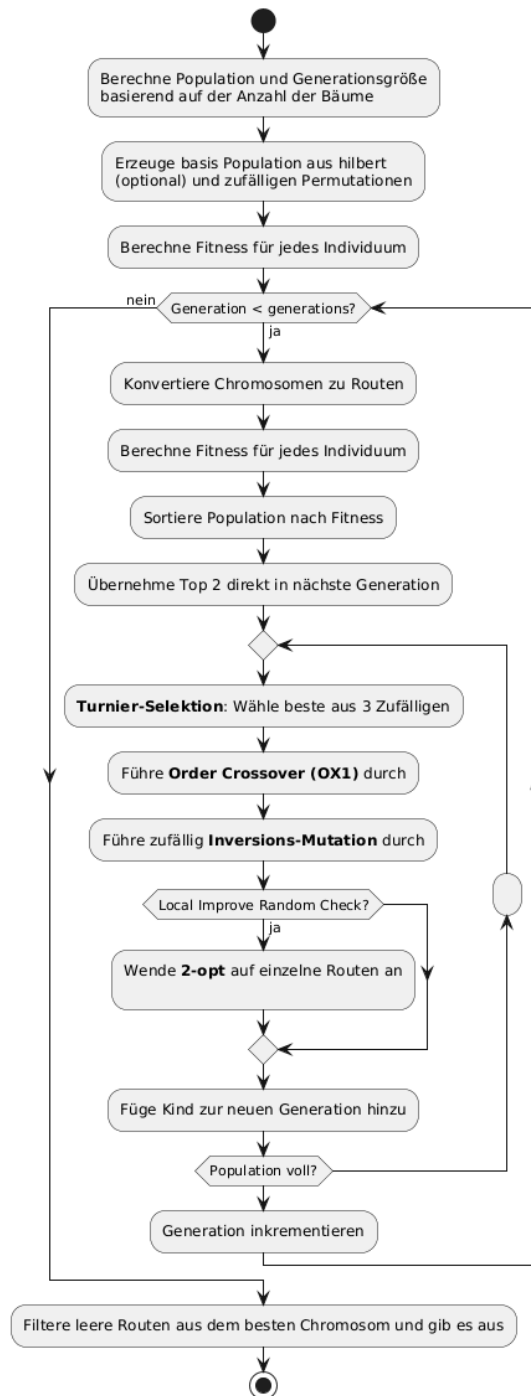


Abbildung 7: Abgewandelter Ablauf eines GA

6.2 Simulierte Abkühlung (Optimierung 2)

6.2.1 Grundlagen

Die Simulierte Abkühlung ist eine stochastische Metaheuristik, die das physikalische Prinzip des kontrollierten Abkühlens von Metallen nachbildet. Der Kern des Verfahrens liegt im Metropolis-Kriterium, das es erlaubt, neben Verbesserungen auch Verschlechterungen der Lösung mit einer temperaturabhängigen Wahrscheinlichkeit zu akzeptieren:

$$P(\Delta E) = \exp\left(-\frac{\Delta E}{T}\right) \quad (2)$$

[7]

Dabei beschreibt:

- $\Delta E = E_{neu} - E_{alt}$ die Energiedifferenz zwischen dem neuen und dem aktuellen Zustand,
- T die aktuelle Temperatur des Systems

Die Temperatur steuert dabei das Gleichgewicht zwischen Exploration (Erkundung des gesamten Lösungsraums) und Exploitation (Feinoptimierung lokaler Optima):

- Bei hoher Temperatur werden auch größere Verschlechterungen akzeptiert, um lokale Optima zu verlassen
- Mit sinkender Temperatur sinkt die Akzeptanzwahrscheinlichkeit für Verschlechterungen kontinuierlich. Der Algorithmus fokussiert sich auf die Feinoptimierung der aktuell besten Lösung und konvergiert gegen Ende in eine lokal stabile, qualitativ hochwertige Lösung [7]

Der Algorithmus terminiert, sobald eine vordefinierte Minimaltemperatur unterschritten wird, und gibt die während des gesamten Prozesses gefundene beste Lösung zurück.

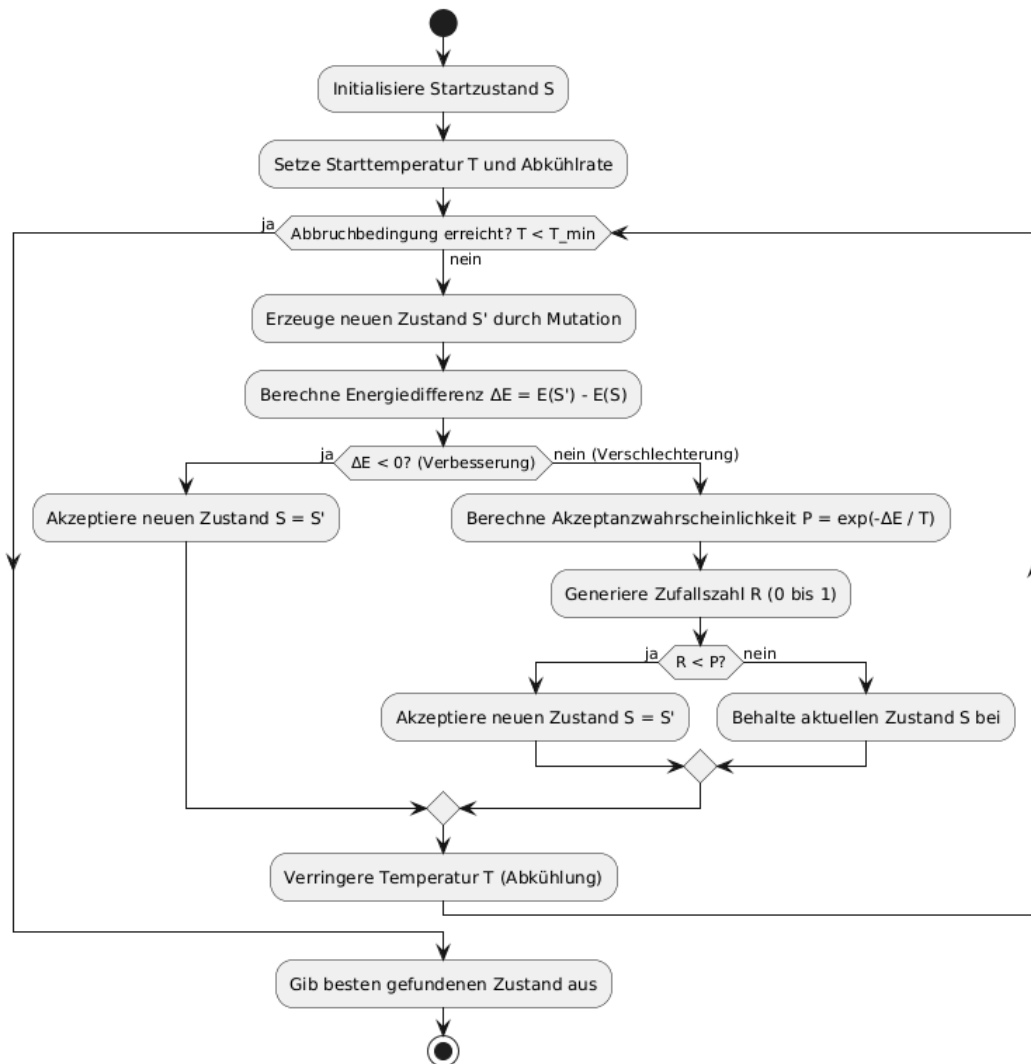


Abbildung 8: Ablauf eines SA

6.2.2 Energieberechnung

Die Qualität einer Lösung (Zustand) wird über eine Energiefunktion bewertet. Je niedriger der Energiewert, desto besser ist die Lösung. Die Funktion ist an die zweistufige Zielsetzung der Aufgabenstellung angepasst und lautet wie folgt:

$$E(S) = \alpha \cdot k + \beta \cdot \frac{T_0}{T_i} \cdot \sum_{r \in S} \max(0, (l(r) - L_{\max})^2) + \gamma \cdot \sum_{r \in S} l(r) \quad (3)$$

- α : Gewichtung zur Minimierung der Roboteranzahl k
- β : Gewichtung von Routen, die $L_{\max}$ überschreiten
- γ : Gewichtung zur Reduzierung der Gesamtdistanz

Die einzelnen Terme und ihre Gewichtung werden im Folgenden detailliert erläutert:

1. Optimierungsziel 1: Minimierung der Roboteranzahl $\alpha \cdot k$.

k beschreibt die Anzahl der Routen (Roboter) in der Lösung S . Die Gewichtungsfaktoren α und β sind von deutlich höher gewählt als γ ($\alpha \geq \beta \geq \gamma$). Dies stellt sicher, dass der Algorithmus vorrangig die Anzahl der benötigten Roboter minimiert.

2. Optimierungsziel 2: Minimierung der Gesamtfahrstrecke $\gamma \cdot \sum_{r \in S} l(r)$

Der Gewichtungsfaktor γ sorgt dafür, dass bei gleicher Anzahl an Robotern die Lösung mit der kürzeren Gesamtfahrstrecke bevorzugt wird. Dies steigert die Praxistauglichkeit der generierten Routen.

3. Nebenbedingung: Strafterm für überlange Routen

Der Term bestraft Routen, die die maximal zulässige Fahrstrecke $L_{\max}$ überschreiten, mit einer quadratischen Strafe, um die Auswirkung von Verstößen stark zu gewichten. Der Faktor $\frac{T_0}{T_i}$ mit der Starttemperatur T_0 und der aktuellen Temperatur T_i sorgt für eine dynamische Anpassung der Strafe. Je tiefer die Temperatur, desto stärker ist der Straffaktor, wodurch gegen Ende valide Lösungen stark bevorzugt werden. Die finale Einhaltung aller Randbedingungen wird anschließend durch eine explizite Validierungsprüfung sichergestellt.

6.2.3 Zustandsveränderung (Nachbarschaftsoperatoren)

Um einen neuen Nachbarzustand aus der aktuellen Lösung zu generieren, werden vier verschiedene Mutationsoperatoren eingesetzt. Ihre Auswahlwahrscheinlichkeit passt sich linear an den Fortschritt des Abkühlprozesses an, mittels des Faktors $Fortschritt = \frac{T_i}{T_0}$:

$$P(\text{Operator}) = 0.1 + \text{Startwert} \cdot \text{Fortschritt} \quad (4)$$

Dadurch werden am Anfang des Prozesses bevorzugt Operationen durchgeführt, die die globale Struktur der Lösung verändern, während gegen Ende nur noch lokale Feinoptimierungen vorgenommen werden.

1. **Relokation (Startwahrscheinlichkeit 0.4):** Ein zufälliger Baum wird aus seiner aktuellen Route entfernt und an einer zufälligen Position in eine andere Route eingefügt. Falls die Ursprungsrouten dadurch leer wird, wird sie automatisch entfernt.
2. **Swap (Startwahrscheinlichkeit 0.3):** Zwei zufällig ausgewählte Bäume aus unterschiedlichen Routen tauschen ihre Positionen.
3. **Shuffle (Startwahrscheinlichkeit 0.2)** Die Reihenfolge einiger Bäume innerhalb einer zufällig ausgewählten Route wird zufällig neu gemischt.
4. **Inversion (2-Opt, Restwahrscheinlichkeit)** Ein zusammenhängender Teilabschnitt einer Route wird in seiner Reihenfolge umgedreht.

Zur Optimierung wurde die Nachbarschaftsoperation so angepasst, dass statt einer vollständigen Mutation nur die tatsächlich betroffenen Routen verändert und zwischengespeichert werden. Dadurch werden bei größeren Beispielen mit mehreren hundert Routen kleinere Berechnungen eingespart.

6.2.4 Delta Berechnung

Bei der Berechnung der Energiewerte wird nur die Differenz berechnet, da dies unnötige Wiederberechnungen von bereits berechneten Routenlängen verhindert. Statt einer vollständigen Neuberechnung gilt also:

$$\Delta E = E_{\text{neu}} - E_{\text{alt}} \quad (5)$$

Da die Nachbarschaftsoperation maximal zwei Routen verändert und die Energiefunktion auf Summanden basiert, gilt:

$$\Delta E = \alpha \cdot \Delta k + \beta \cdot \Delta \left(\frac{T_0}{T_i} \cdot \sum_{r \in S} \max(0, (l(r) - L_{\max})^2) \right) + \gamma \cdot \Delta \left(\sum_{r \in S} l(r) \right) \quad (6)$$

Dadurch müssen nur die Vorher- und Nachherwerte der betroffenen Routen berechnet werden, um die Energiedifferenz zu bestimmen.

Außerdem wird für das Metropolis-Kriterium ebenfalls nur das ΔE benötigt. Die Akzeptanzwahrscheinlichkeit einer Verschlechterung lautet:

$$P = \exp \left(-\Delta \frac{E}{T_i} \right) \quad (7)$$

Damit kann jeder Nachbarzustand effizient bewertet werden, ohne die gesamte Lösung in jeder Iteration vollständig neu auszuwerten.

6.2.5 Abkühlung

Für dieses Problem wurde eine exponentielle Abkühlung gewählt. Die Abkühlrate wird dabei an die Problemgröße angepasst:

$$\text{cooling_rate} = \left(\frac{T_{\min}}{T_{\max}} \right)^{\frac{1}{N \cdot I}} \quad (8)$$

Dabei beschreiben:

- $T_{\max}$ bzw. formell T_0 die Starttemperatur
- $T_{\min}$ bzw. formell T_n die Minimaltemperatur (Abbruchbedingung)
- I die Anzahl der Iterationen pro Baum

Die berechnete Abkühlrate wird zusätzlich auf den Bereich $[0.999, 0.999999999]$ begrenzt. Dies verhindert zum einen eine zu schnelle Abkühlung, die zu einer vorzeitigen Konvergenz führen kann, und zum anderen eine zu langsame Abkühlung, die Gleitkomma-Ungenauigkeiten und unnötig lange Laufzeiten verursacht.

6.2.6 Parameter

Die Optimierung der Gewichtungparameter α , β , γ und die maximale Anzahl an Iterationen ohne Verbesserung erfolgte über eine systematische Testreihe. Für die Untersuchung wurden folgende Wertebereiche evaluiert:

SA_ALPHA_VALUES = [300, 500, 800, 1000]

SA_BETA_VALUES = [100, 300, 400, 600]

SA_GAMMA_VALUES = [1, 3, 5, 7]

SA_MAX_ITERATION_VALUES = [208, 416, 833]

SA_ITERATION_VALUES = [1000, 2500, 5000, 25000, 100000]

Auch hier ist die Distanz bewusst nur schwach gewichtet, da die getesteten γ -Werte klein gegenüber den führenden Termen bleiben. Damit priorisiert SA zuerst Roboter Menge und Strafterm für Streckenverletzungen; die Distanz wirkt nur feinjustierend.

Als Testinstanzen dienten die Dateien roboter04, 07, 09, 10 und die selbst-generierte Instanz roboter13 (auch in der Abgabe zu finden). Um die stochastischen Schwankungen des SA zu kompensieren, wurde jede Parameterkombination pro Datei dreimal durchlaufen. Die Parameteroptimierung führte zu folgenden Ergebnissen:

Eingabekomplexität	Anzahl der Bäume (N)	Maximale Routenlänge (L)	Beste Parameterkombination
Niedrig	400	300	alpha=300, beta=100, gamma=1, iterations=10.000, maxIterations=208
Mittel	4.996	300	alpha=300, beta=300, gamma=3, iterations=25.000, maxIterations=416
Hoch	5.000	500	alpha=300, beta=400, gamma=5, iterations=100.000, maxIterations=833

(Die Analyse wurde mithilfe von KI durchgeführt, da die Datei mit 9600 Testfällen für eine manuelle Auswertung zu groß war. Die KI wurde hier zur Mustererkennung und zur strukturierten Voranalyse eingesetzt.)

Final wurde in der Abgabe eine mittlere Konfiguration definiert, die für alle Dateien verwendet wird:

SA_ALPHA = 300

SA_BETA = 300

SA_GAMMA = 3

SA_ITERATIONS = 25000

SA_MAX_ITERATIONS = 416

Die Ergebnisse der Tests sind auch in der Abgabe zu finden. Der Code (SAparam.py) zum Testen und die Analyse der Daten wurde mithilfe von KI gemacht, während die finale Auswertung und Interpretation eigenständig erfolgten.

Daraus erschließt sich, dass das Festlegen der Werte nicht von Nutzen ist, da sich die besten Parameter mit der Komplexität des Problems verändern. Da der mittlere Fall ein Mittelpunkt zwischen dem niedrigen und dem hohen Fall ist und die meisten Parameter teilweise übereinstimmen werden die Werte aus mittel verwendet.

Hier fiel in späteren Tests auf, dass die Parameter nicht optimal waren und höhere Werte für α zu besseren Ergebnissen führten, da sie die Anzahl der Roboter stärker priorisieren, was das Hauptziel der Aufgabenstellung ist, jedoch wurden die Werte aus der Testreihe beibehalten, da diese durch systematische Tests ermittelt wurden und daher nachvollziehbar sind, während die späteren Anpassungen eher auf subjektiven Beobachtungen basieren.

6.2.7 Wahl der Heuristik

Die Entscheidung für die SA als zusätzliche Heuristik basiert auf folgenden Kernpunkten:

- **Effizienz bei NP-schweren Problemen:** Das MDEVPR ist für größere Instanzen praktisch nicht mehr exakt lösbar. SA kann hier in polynomieller Laufzeit oft hochwertige Näherungslösungen liefern [8].
- **Einfache Implementierung:** Ein Vorteil ist die vergleichsweise klare Logik und die dadurch meist gut nachvollziehbare Implementierung gegenüber stärker populationsbasierten Ansätzen.
- **Umgang mit lokalen Optima:** Durch das Metropolis-Kriterium kann das Verfahren lokale Optima besser verlassen, da vorübergehende Verschlechterungen zugelassen werden [8].
- **Flexibilität der Zielvorgaben:** Über die gewichtete Energiefunktion lässt sich die Priorisierung (Roboteranzahl vor Routenlänge) gezielt einstellen.
- **Adaptive Exploration:** Durch die Anpassung der Nachbarschaftsoperatoren (Relocation, Swap, Shuffle, 2-Opt) kann der Übergang von globaler Suche zu lokaler Feinoptimierung unterstützt werden.
- **Stabilität:** In den durchgeführten Tests zeigte SA teils eine robuste Konvergenz und war in einzelnen Fällen weniger anfällig für vorzeitigen Stillstand durch mangelnde Diversität [8].

6.2.8 Implementierung

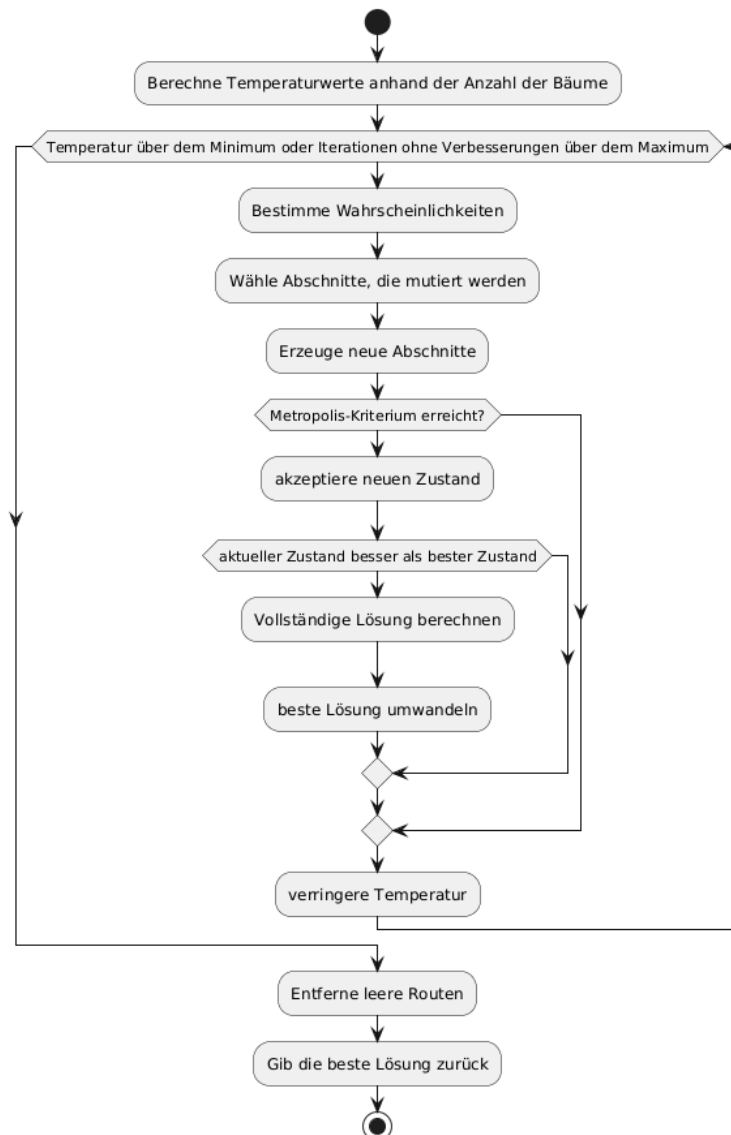


Abbildung 9: Abgewandelter Ablauf des SA

Die Implementierung der Simulierten Abkühlung wurde speziell an die Anforderungen des Gießroboter-Problems angepasst und weist gegenüber einer Standardimplementierung mehrere Erweiterungen auf:

- Die Abkühlrate wird an die Anzahl der Bäume in der Eingabedatei angepasst. Dadurch wird sichergestellt, dass der Algorithmus sowohl bei kleinen als auch bei sehr großen Dateien eine ausreichende Zeit zur Lösungsfindung hat.
- Die Mutation bzw. der Nachbarschaftsoperator agieren nur auf lokaler Basis, anstatt die gesamte Lösung zu verändern. Dadurch wird die Laufzeit, wegen weniger Kalkulationen und Veränderungen, verringert und es ermöglicht mehr Iterationen pro Baum durchzuführen. Die vollständige Veränderung wird nur bei der Speicherung der Lösung durchgeführt.
- Ähnlich wie bei dem Nachbarschaftsoperator wird bei der Energieberechnung nur ein Δ berechnet, wobei nur die Energie der veränderten Routen berechnet wird und die Energie der Version der Routen davor wird mit dieser subtrahiert um ein ΔE zu berechnen, wodurch das Berechnen, von bereits bekannten Routen, verhindert wird. Die vollständige Energie wird nur bei der Initialisierung zur Definition einer Start- bzw. Basisenergie verwendet.
- Die Auswahlwahrscheinlichkeit der vier Mutationsoperatoren ist abhängig von der aktuellen Temperatur. Dies optimiert das Gleichgewicht zwischen globaler Umstrukturierung und lokaler Feinoptimierung.

- Es wird die aktuell beste gefundene Lösung statt der jetzigen Lösung ausgegeben, wobei leere Routen automatisch entfernt werden.
- Es wurde eine weitere Abschlussbedingung eingeführt, falls nach mehreren Iterationen kein neues Minimum erreicht wird, wird das Programm auch frühzeitig beendet

Als Erweiterung wäre die Integration eines memetischen Lokalsuchverfahrens denkbar: Am Ende des Abkühlprozesses könnte eine 2-Opt-Optimierung auf alle Routen angewendet werden, um die Gesamtfahrstrecke der finalen Lösung nochmals zu reduzieren.

Im Gegensatz zum Standard-SA-Verfahren implementiert SA_Improver.py eine inkrementelle Delta-Berechnung. Bei Relocate oder Swap werden nur die Längen der zwei betroffenen Routen aktualisiert. Dies reduziert die Komplexität pro Iteration von einer vollständigen Neuberechnung über alle Routen auf lokale Neuberechnungen der betroffenen Routen und reduziert so den praktischen Laufzeitaufwand deutlich, insbesondere bei großen Instanzen (z. B. roboter11).

6.3 Finale Parametrisierung

Für die Abgabe wird daher die mittlere Konfiguration als Standard verwendet. Sie bildet den besten Kompromiss zwischen Qualität, Stabilität und Laufzeit.

Bereich	Aktive Werte	Bedeutung
GA	alpha = 500, beta = 600, gamma = 1, mutation_prob = 0.01, population = 50, generations = 50, local_improve_prob = 0.15	Standardkonfiguration für den Genetischen Algorithmus; angepasst an mittlere Instanzen und reproduzierbare Laufzeiten.
SA	alpha = 300, beta = 300, gamma = 3, iterations = 25.000, maxIterations = 416	Standardkonfiguration für die Simulierte Abkühlung; ausgewogenes Verhältnis aus Exploration und Strafe für Überlängen.
Determinismus	SA_RANDOM_SEED = 4, GA_RANDOM_SEED = 4	Feste Seeds machen die Ergebnisse für Vergleiche besser reproduzierbar.

7 Umsetzung des Programms

Das Programm wurde in Python 3.11.x implementiert und verfolgt einen modularen Aufbau, um das Testen und Erweitern des Programms zu vereinfachen.

Nutzung:

```
python main.py --filepath <Eingabedatei> [--algorithm 1|2|3] [--plot]
```

Argumente:

- --filepath: Pfad zur Eingabedatei.
- --algorithm (optional, standard: 3): Auswahl des Verbesserungsalgorithmus
 - 1 = Hilbert (nur Initiallösung)
 - 2 = Hilbert + Simulated Annealing
 - 3 = Hilbert + Genetic Algorithm
- --sa-restarts (optional, standard: 1): Anzahl der SA-Wiederholungen mit festen Seeds.
- --ga-restarts (optional, standard: 1): Anzahl der GA-Wiederholungen mit festen Seeds.
- --plot (optional, standard: False): Wenn True, werden Initial- und Endlösung geplottet.

Beispiele:

1. Nur Initiallösung

```
python main.py --filepath <Eingabepfad> --algorithm 1
```

2. Simulated Annealing (Standard)

```
python main.py --filepath <Eingabepfad> --algorithm 2
```

3. Genetic Algorithm

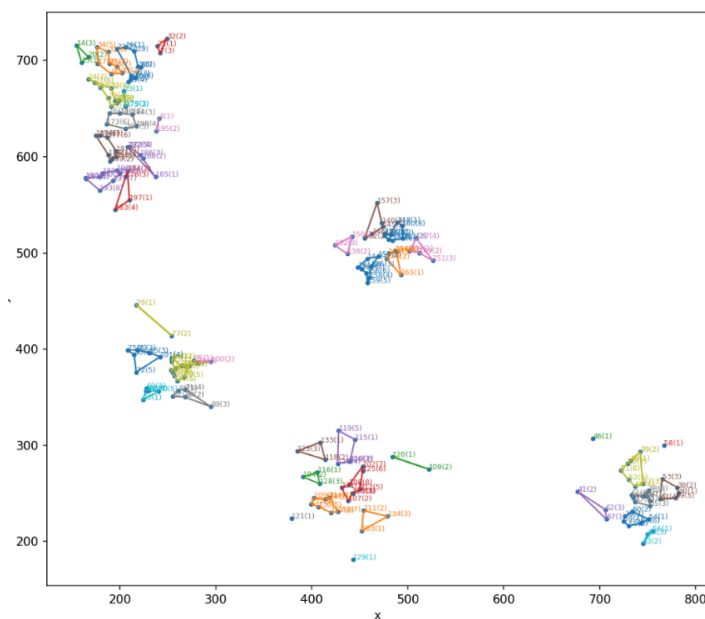
```
python main.py --filepath <Eingabepfad> --algorithm 3
```

4. Mit Plot (Visualisierung)

```
python main.py --filepath <Eingabepfad> --plot
```

Abbildung 10 Beispiel Plot für roboter04

Beispielplot von roboter04 mit der Hauptlösung (Hilbert-Greedy, mit SA optimiert)



Unterschiedliche Farben stehen für die Routen der jeweiligen Roboter. Das Label im Format „ID (Besuchsreihenfolge)“, gibt die ID des Baumes sowie die Reihenfolge seines Besuchs in der Route an.

7.1 Bibliotheken

Folgende externe Bibliotheken werden verwendet:

Bibliothek	Anwendung	Version
random	wird zur Erzeugung stochastischer Prozesse eingesetzt	Standardbibliothek
math	wird für mathematische Operationen verwendet	Standardbibliothek
argparse	dient zur Verarbeitung von Kommandozeilenargumenten	Standardbibliothek
pathlib	überprüft die Validität von Dateipfaden	Standardbibliothek
time	wird zur Laufzeitmessung eingesetzt	Standardbibliothek
matplotlib.pyplot	dient zur Visualisierung der erzeugten Lösungen. (nicht gefordert vereinfacht die Auswertung der Ausgaben)	3.10.8
numpy	wird zur effizienten Speicherung und Verarbeitung der Distanzmatrix verwendet	2.4.3

7.2 Dateien

Aufteilung:

- `config.py`: zentrale Hyperparameter für Hilbert, SA und GA.
- `initialize.py`: liest Eingaben ein und berechnet die Distanzmatrix.
- `general.py`: zentrale Hilfsfunktionen (z. B. Distanz und Routenlänge).
- `Hilbert.py`: erzeugt eine räumlich sinnvolle Reihenfolge der Bäume.
- `Greedy.py`: baut aus der Hilbert-Reihenfolge eine gültige Initiallösung.
- `GA_Improver.py`: verbessert die Initiallösung mit genetischer Suche.
- `SA_Improver.py`: verbessert die Initiallösung mit Simulated Annealing.
- `validitycheck.py`: prüft Lösungsgültigkeit (Vollständigkeit, Distanzgrenzen, Eindeutigkeit).
- `main.py`: steuert den Gesamtprozess und wählt den Optimierungsmodus.

Ergänzende Dateien (`result_printer.py`, `plotter.py`, `log.py`) dienen Ausgabe, Visualisierung und Protokollierung.

7.2.1 `config.py`

Zentrale Datei zur Steuerung der Heuristiken mittels Konstanten (Floats/Integers).

Hilbert:

- **HILBERT_MARGIN**: Erhöht die Auflösung (Standard: 3).

Simulierte Abkühlung:

- **Temperatur**: Start (SA_TMAX:10000.0) und Endtemperatur (SA_TMIN 0.01).
- **Cooling Rate**: Bereich zwischen 0.999 (MIN) und 0.999999999 (MAX) für numerische Stabilität.
- **Gewichtung**: Parameter ALPHA, BETA, GAMMA (300, 300, 3) zur Bewertung von Roboteranzahl, Überlängen und Gesamtdistanz.
- **Iterationen**: SA_ITERATIONS (25000) pro Baum; SA_MAX_ITERATIONS (Faktor 416) begrenzt die Suche ohne Verbesserung.

Genetischer Algorithmus:

44. Bundeswettbewerb Informatik - 2. Runde - Aufgabe A2: GießroboterTeilnahme-ID: 79280

Dynamische Konfiguration basierend auf der Gartengröße: Small (≤ 333 Bäume), Medium (≤ 1926) und Large (> 1926).

- **Mutation & Lokale Suche:** `mutation_prob` zur Diversitätssicherung und `local_improve_prob` (0.15) für memetische Optimierung.
- **Population & Generationen:** Gesteuert durch `standard_population` (50) und limitiert durch `max_population/max_generations` (500).
- **Gewichtung:** Variable Parameter (Alpha, Beta, Gamma) je nach Gartenkategorie.
- **Routen-Toleranz:** `overlength_margin` definiert den Spielraum für Routenüberschreitungen vor der Trennung durch den Greedy-Algorithmus.

7.2.2 initialize.py

Initialisierungscode zur Aufbereitung der Rohdaten in strukturierte Variablen.

1.readfile(path)

Beschreibung: Liest BWINF-konforme Dateien ein und extrahiert die maximale Routenlänge ($L_{\max}$), die Baum-Anzahl sowie die Koordinaten der Bäume.

Validierung (Failsafes):

- **Datentypen:** Prüfung, ob $L_{\max}$, Baum-Anzahl und Koordinaten gültige Ganzzahlen sind.
- **Wertebereich:** $L_{\max}$ und die Anzahl der Bäume müssen > 0 sein.
- **Vollständigkeit:** Abgleich der tatsächlichen Zeilenanzahl mit der angekündigten Baum-Anzahl.
- **Sequenzierung:** Sicherstellung, dass Baum-IDs lückenlos sequenziell (1 bis N) verlaufen, um die direkte Indexierung in Numpy-Arrays zu ermöglichen.

Parameter & Rückgabe:

- **Eingabe:** `path(string)`
- **Ausgabe:** `trees(dict: ID → Koord.)`, `ids(list)`, `maxl(int)`

2. precompute_dist_matrix(ids, trees, dtype=np.float32)

Beschreibung: Berechnet eine vollständige Distanzmatrix mittels **Numpy-Broadcasting**. Dies reduziert die Komplexität von $O(n^2 \cdot k)$ auf $O(n^2)$ und minimiert redundante Berechnungen während der Laufzeit.

Logik: Durch Vektorisierung werden Differenz-Tabellen für x- und y-Koordinaten erstellt:

$$dx = x_i - x_j, \quad dy = y_i - y_j \quad (9)$$

Die finale Matrix resultiert aus der Anwendung des Satzes des Pythagoras ($dx^2 + dy^2$).

Parameter & Rückgabe:

- **Eingabe:** `ids`, `trees`, `dtype(Standard: float32)`
- **Ausgabe:** `dist_matrix(2D np.ndarray)`

7.2.3 general.py

Enthält zentrale Hilfsfunktionen zur Distanzberechnung und Routenanalyse.

distance(a_id, b_id, dist_matrix, use_sqrt=True)

Beschreibung: Ruft die Distanz zwischen zwei Punkten basierend auf ihren IDs aus der Matrix ab.

- **Logik:** Greift auf `dist_matrix[a_id-1][b_id-1]` zu.
- **Optimierung:** Durch `use_sqrt=False` können quadrierte Distanzen für Vergleiche genutzt werden, was Rechenzeit spart und Rundungsfehler vermeidet.
- **Parameter:** `a_id, b_id(IDs)`, `dist_matrix(Array)`, `use_sqrt(Boolean)`.
- **Rückgabe:** Euklidische Distanz ($\sqrt{d^2}$) oder quadrierte Distanz (d^2).

route_length(route, real_distmatrix)

Beschreibung: Berechnet die Gesamtlänge einer gegebenen Route durch Summation der Teilstrecken in der Distanzmatrix.

- **Failsafe:** Gibt bei einer leeren Route sofort 0.0 zurück.
- **Anwendung:** Dient als Basis für die Fitness- und Energiefunktionen der Optimierungsalgorithmen, um die Güte eines Zustands oder Chromosoms zu bewerten.
- **Parameter:** route(Liste von Baum-IDs), real_distmatrix(Matrix mit bereits radizierten Werten).
- **Rückgabe:** Gesamte Streckenlänge (float).

7.2.4 Hilbert.py

Implementierung der Hilbert-Kurve zur räumlichen Sortierung der Bäume, um eine effiziente Reihenfolge für den Greedy-Algorithmus zu generieren.

xy2d(n, x, y)

Beschreibung: Berechnet den Hilbert-Index eines Punktes (x, y) in einem Gitter der Ordnung n .

- **Logik:** Der Algorithmus durchläuft die Ebenen der Kurve iterativ, bestimmt den Quadranten des Punktes, addiert den entsprechenden Distanzwert und rotiert das Koordinatensystem für die nächste Iteration.
- **Laufzeit:** $O(\log N)$, da die Schleife pro Halbierungsschritt ($s \geq 1$) einmal ausgeführt wird.
- **Parameter:** n(Gittergröße), x, y(Koordinaten).
- **Rückgabe:** d(Hilbert-Index).

hilbert_sort(trees, ids)

Beschreibung: Sortiert alle Bäume basierend auf ihrer Position entlang der Hilbert-Kurve.

- **Ablauf:**
 1. **Normalisierung:** Ermittelt die Ausdehnung des Feldes und skaliert die Koordinaten auf ein Einheitsquadrat (Gitter), um die Hilbert-Berechnung zu ermöglichen.
 2. **Transformation:** Berechnet für jeden Baum mittels xy2d den Hilbert-Index.
 3. **Sortierung:** Erstellt ein Ranking der Baum-IDs, sortiert nach ihren Indizes.
- **Vorteil:** Erhält die räumliche Nähe der Punkte besser als eine einfache Sortierung nach X- oder Y-Koordinaten.
- **Laufzeit:** $O(N \log N)$, resultierend aus N Transformationen und der anschließenden Sortierung.
- **Eingabe:** trees(Koordinaten-Mapping), ids(ID-Liste).
- **Ausgabe:** Liste der Baum-IDs in räumlich sortierter Reihenfolge.

7.2.5 Greedy.py

Implementierung eines distanzbasierten Greedy-Algorithmus zur Erzeugung einer validen Initiallösung für die nachfolgenden Metaheuristiken.

greedy(hilbert, maxdist, distmatrix)

Beschreibung: Erzeugt Routen, indem die räumlich sortierte Liste (hilbert) sequenziell abgearbeitet wird.

- **Logik:** Bäume werden der aktuellen Route hinzugefügt, solange die Gesamtlänge (inkl. Rückweg zum Depot) $L_{\max}$ nicht überschreitet. Bei Überschreitung wird eine neue Route begonnen.
- **Failsafes:** - Stellt sicher, dass jede Route mindestens einen Baum enthält (vermeidet Index-Fehler).
 - Hängt verbleibende Bäume nach Abschluss der Iteration als finale Route an.
- **Laufzeit:** $O(N)$, da jeder Baum genau einmal geprüft wird.

Parameter & Rückgabe:

- **Eingabe:** hilbert (sortierte ID-Liste), maxdist (limitierende Streckenlänge), distmatrix (Distanztabelle).
- **Ausgabe:** routes (Liste von Listen mit Baum-IDs).

7.2.6 GA_Improver.py

Dieses Modul implementiert einen **memetischen Algorithmus** (Genetischer Algorithmus kombiniert mit lokaler Suche), um die Initiallösung durch Evolution und gezielte Mutationen zu optimieren.

Konfiguration und Vorbereitung

- **Adaptive Skalierung:** `apply_size_based_ga_config` und `calculate_population_generations` passen Populationsgröße und Generationenanzahl dynamisch an die Baum-Anzahl (N) an.
- **Effizienz:** `ensure_real_dist_matrix` präcomputiert die euklidischen Distanzen ($\sqrt{d^2}$) in $O(N^2)$, um wiederholte Wurzelberechnungen im Hauptloop zu vermeiden.
- **Struktur:** `flat` wandelt 2D-Routenlisten in 1D-Chromosomen (Permutationen) um ($O(N)$).
- **Seed-Listen:** `GA_FIXED_RESTART_SEEDS` gibt die feste Reihenfolge der Startseeds vor. Bei mehreren Wiederholungen werden diese Seeds der Reihe nach abgearbeitet.

`unflatten_and_score(...)`

Wandelt ein flaches Chromosom mittels eines erweiterten Greedy-Ansatzes in valide Routen zurück.

- **Modus `solution=True`:** Erzeugt strikt valide Routen für die finale Ausgabe.
- **Modus `solution=False`:** Erlaubt eine definierte Überschreitung (`GA_OVERLENGTH_MARGIN`), um die genetische Vielfalt zu erhalten und vorzeitige Konvergenz zu verhindern.

`calculate_fitness(...)`

Berechnet die Güte eines Chromosoms. Die Fitnessfunktion F minimiert die Anzahl der Roboter (k), die Gesamtdistanz und bestraft Längenüberschreitungen:

$$F = \alpha \cdot k + \beta(g) \cdot \sum \max(0, l(r) - L_{\max})^2 + \gamma \cdot \sum l(r) \quad (10)$$

Der Strafterm $\beta(g)$ steigt mit fortschreitender Generation g an, um die Lösung zunehmend in den zulässigen Bereich zu zwingen.

Genetische Operatoren

- **Crossing (OX1):** `orderx` kombiniert zwei Eltern, indem ein Segment von Parent 1 übernommen und die restlichen Gen-Positionen durch Parent 2 in deren relativer Reihenfolge aufgefüllt werden ($O(N)$).
- **Mutation:** `mutate` führt eine Inversionsmutation durch (invertiert ein zufälliges Teilsegment), um die Diversität zu erhöhen.
- **Lokale Suche (2-Opt):** `local_improve_vrpo` optimiert die Besuchsreihenfolge innerhalb einzelner Routen durch Kanten-Swaps ($O(k^2 \cdot I)$). Dies bildet die memetische Komponente.

`ga_improve(base_routes, ...)`

Der evolutionäre Hauptloop:

1. **Initialisierung:** Erzeugt die Startpopulation aus der Greedy-Lösung und diversifizierten Varianten (Mutation/Shuffle).
2. **Evolution:** Pro Generation werden mittels **Turnierselektion** Eltern gewählt, Nachkommen durch OX1/Mutation erzeugt und optional lokal verbessert.
3. **Elitismus:** Die zwei besten Individuen werden direkt in die nächste Generation übernommen.
4. **Abschluss:** Ein finaler 2-Opt-Durchlauf mit hoher Iterationszahl maximiert die Güte der Endlösung.

Laufzeitkomplexität: $O(G \cdot P \cdot q \cdot I \cdot N^2)$ (Abhängig von Generationen G , Population P , Wahrscheinlichkeit der lokalen Suche q und Iterationen I).

7.2.7 SA_Improver.py

Implementierung der **Simulated Annealing** (SA) Metaheuristik zur stochastischen Optimierung der Routen durch Modellierung eines physikalischen Abkühlungsprozesses.

`coolingr(num_points, iterations, cooling_rate)`

Beschreibung: Berechnet adaptive Temperaturparameter für eine kontrollierte Transition von Exploration zu Exploitation.

- **Logik:** Bestimmt die Abkühlungsrate c so, dass $T_{\max}$ über I Schritte auf $T_{\min}$ sinkt: $c = \left(\frac{T_{\min}}{T_{\max}}\right)^{\frac{1}{I}}$.
- **Failsafe:** Begrenzt die Rate durch `SA_MAX_COOLING_RATE`, um die numerische Stabilität zu garantieren ($O(1)$).

`energy(solution, maxdist, temperature, distmatrix)`

Beschreibung: Quantifiziert die Qualität einer Lösung (niedriger ist besser).

- **Komponenten:**

1. Anzahl der Routen k (Gewichtung α).
2. Strafterm für Überlängen: Steigt antiproportional zur Temperatur ($\frac{\beta}{T}$). Dies erlaubt anfangs Grenzüberschreitungen zur Vermeidung lokaler Optima.
3. Gesamtdistanz aller Routen (γ).

- **Laufzeit:** $O(N)$, da jede Route der Lösung einmal summiert wird.

neighbour(solution, temperature, init_temperature, ...)

Beschreibung: Erzeugt eine neue Lösung durch stochastische Modifikation (Nachbarschaftssuche).

- **Operatoren:**

- **Relocate:** Verschiebt einen Baum in eine andere Route.
- **Swap:** Tauscht Bäume zwischen zwei Routen.
- **Modify/Shuffle:** Interne Umstrukturierung einer einzelnen Route.

- **Dynamik:** Mit sinkender Temperatur verschiebt sich der Fokus von globalen Strukturänderungen hin zu lokalen Optimierungen ($O(k)$).

sa_improve(coords, base, maxdist, distmatrix, ...)

Beschreibung: Hauptloop der SA-Optimierung.

- **Ablauf:** Ausgehend von der base-Lösung werden iterativ Nachbarn erzeugt. Verschlechterungen werden mit der Wahrscheinlichkeit $P = \exp(-\Delta \frac{E}{T})$ akzeptiert (Metropolis-Kriterium).
- **Abbruch:** Endet bei Erreichen von min_temp oder wenn über max_no_improve Iterationen keine Verbesserung erzielt wurde.
- **Laufzeit:** $O(I \cdot N)$, linear zur Anzahl der Iterationen.
- **Seed-Listen:** SA_FIXED_RESTART_SEEDS definiert die feste Reihenfolge der Startseeds für wiederholte SA-Durchläufe.

7.2.8 validitycheck.py

Dient der abschließenden Verifikation der berechneten Lösungen vor der Ausgabe.

validity_check(routes, distmatrix, maxdist, node_ids)

Beschreibung: Prüft die Integrität der Lösung gegen die Problemstellung.

- **Prüfkriterien:**

- **Vollständigkeit:** Jeder Baum aus node_ids muss genau einmal vorkommen.
- **Validität:** Keine leeren Routen, keine unbekannten/doppelten IDs.
- **Restriktion:** Keine Route darf die Maximallänge maxdist überschreiten.

- **Rückgabe:** True bei Erfolg; andernfalls (False, Error-String).

7.2.9 main.py

Das Hauptprogramm koordiniert den Gesamtprozess, von der Dateieinlesung bis zur Ausgabe der optimierten Route.

main()

Beschreibung: Steuert den Programmfluss von der Initialisierung bis zur Ausgabe. Die aktuelle Implementierung arbeitet deterministisch über den gewählten Algorithmusmodus und optionale Restart-Parameter.

- **Ablauf:**

1. Einlesen und Vorverarbeitung via initialize.
2. Erzeugung einer Initiallösung über Hilbert-Sortierung und Greedy.
3. Optionale Verbesserung mit SA oder GA.
4. Validierung und globaler Fallback auf die Initiallösung, falls die Optimierungslösung ungültig ist oder mehr Roboter verwendet.
5. Protokollierung, optionale Visualisierung und Ausgabe der finalen Lösung.

- **Optimierungssteuerung:**

- **Algorithmusauswahl:** --algorithm mit 1 (Hilbert), 2 (SA), 3 (GA).

44. Bundeswettbewerb Informatik - 2. Runde - Aufgabe A2: Gießroboter Teilnahme-ID: 79280

- **Deterministische Restarts:** `--sa-restarts` und `--ga-restarts` verwenden feste Seed-Listen in den jeweiligen Modulen.
- **Fallback:** Ist die Optimierungslösung ungültig oder nutzt mehr Roboter als die Initiallösung, wird auf die Initiallösung zurückgesetzt.

Failsafes:

- **Dateiprüfung:** Verifiziert die Existenz des angegebenen Dateipfads.
- **Validierung:** Stellt sicher, dass die Algorithmuswahl (1, 2 oder 3) gültig ist.
- **Terminierung:** Die SA/GA-Module terminieren intern über ihre jeweiligen Abbruchkriterien und Restart-Grenzen.

8 Verwendung von KI

Künstliche Intelligenz wurde in dieser Bearbeitung ausschließlich als unterstützendes Werkzeug eingesetzt. Die Kerninhalte – die algorithmische Konzeption, die Designentscheidungen für die SA, die Implementierung des Kerncodes, die mathematische Modellierung und die Ergebnisanalyse – wurden vollständig eigenständig erstellt.

KI wurde in folgenden Bereichen eingesetzt:

1. Grundlegende Literaturrecherche zum Thema Vehicle Routing Problem und Simulierte Abkühlung.
2. Erstellung von Code zur automatisierten Durchführung der Parameteroptimierung und zur statistischen Auswertung der Testergebnisse.
3. Unterstützung bei der Formalisierung und Korrektur der Dokumentation.

Alle KI-unterstützten Inhalte wurden anschließend eigenständig überprüft, angepasst und validiert. Die Alternativtexte für die Bilder wurden in Word unbeabsichtigt automatisch mit erzeugt.

Zusätzlich wurde mithilfe von KI eine Erweiterung der aktuellen Implementierung erstellt, um potenzielle Verbesserungen vergleichen und Leistungsunterschiede darstellen zu können. Diese Version ist in der Abgabe unter `solver_numba` enthalten und stellt eine Kopie der aktuellen Implementierung mit entsprechenden KI-gestützten Erweiterungen dar.

9 Beispiele

Für die Ergebnisse wurden alle bwinf Beispieldateien und drei extra Dateien (roboter12.txt, roboter13.txt und roboter14.txt) untersucht

Die drei zusätzlichen Dateien repräsentieren spezielle Randfälle:

- roboter12.txt: 2 Bäume die zu weit weg voneinander sind

10

2

1 100 100

2 0 0

- roboter13.txt: 1 Baum

1000

1

1 1 1

- roboter14.txt: keine Bäume

100

0

Datei	Hilbert	SA	GA
roboter01	4 Roboter (0.0000s)	3 Roboter (3.1687s)	3 Roboter (0.3124s)
roboter02	9 Roboter (0.0000s)	8 Roboter (2.8078s)	9 Roboter (0.3240s)
roboter03	76 Roboter (0.0000s)	71 Roboter (134.0869s)	69 Roboter (22.1054s)
roboter04	47 Roboter (0.0000s)	41 Roboter (48.2514s)	44 Roboter (3.4349s)
roboter05	4 Roboter (0.0009s)	4 Roboter (79.3467s)	4 Roboter (34.6160s)
roboter06	7 Roboter (0.0000s)	7 Roboter (65.5552s)	7 Roboter (7.8281s)
roboter07	220 Roboter (0.2008s)	220 Roboter (1926.1454s)	205 Roboter (4351.7577s)
roboter08	706 Roboter (0.1626s)	704 Roboter (1707.3017s)	690 Roboter (2102.2852s)
roboter09	58 Roboter (0.0210s)	56 Roboter (104.4882s)	56 Roboter (16.9551s)
roboter10	593 Roboter (0.1763s)	591 Roboter (1623.5589s)	585 Roboter (2296.4548s)
roboter11	2395 Roboter (2.2088s)	2394 Roboter (7650.4929s)	2400 Roboter (10287.5559s)
roboter12	2 Roboter (0.0082s)	2 Roboter (0.4588s)	2 Roboter (0.3283s)
roboter13	1 Roboter (0.0059s)	1 Roboter (0.4306s)	1 Roboter (0.0096s)
roboter14	Keine Eingabe → Eingabe wurde in initialize.py gestoppt (0.0000s)		

Tabelle 3: Tabelle 1 Ergebnisvergleich

Bei überkomplexen Graphen und Graphen mit weniger als drei Punkten wurde der Plot entfernt, um die Dokumentation in einem passenden Umfang zu halten.

roboter01.txt

Initiallösung

Ergebnis:

4

175

129

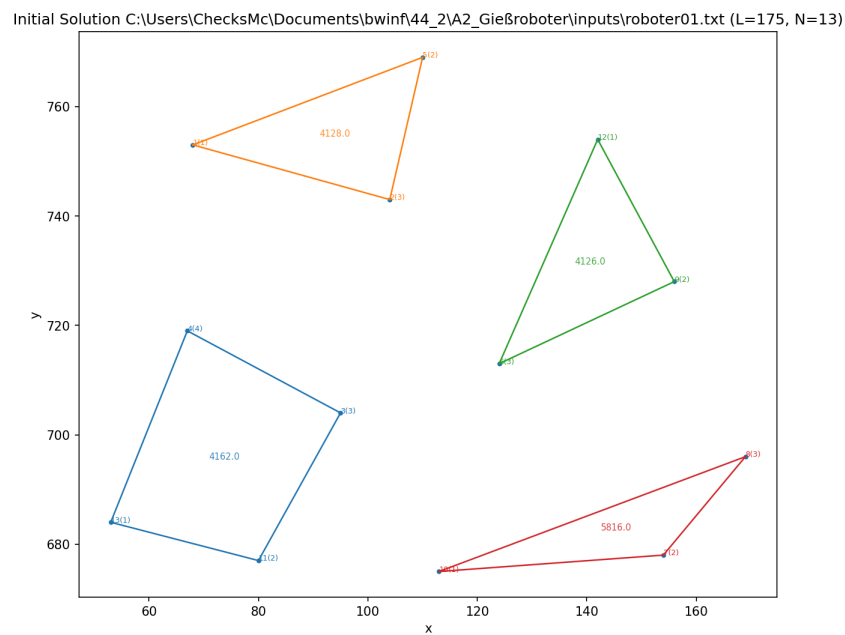
53 684

44. Bundeswettbewerb Informatik - 2. Runde - Aufgabe A2: Gießroboter Teilnahme-ID: 79280

13 11 3 4 13
109
68 753
1 5 2 1
110
142 754
12 9 6 12
125
113 675
10 7 8 10

Laufzeit: 0.0000 s

Plot:



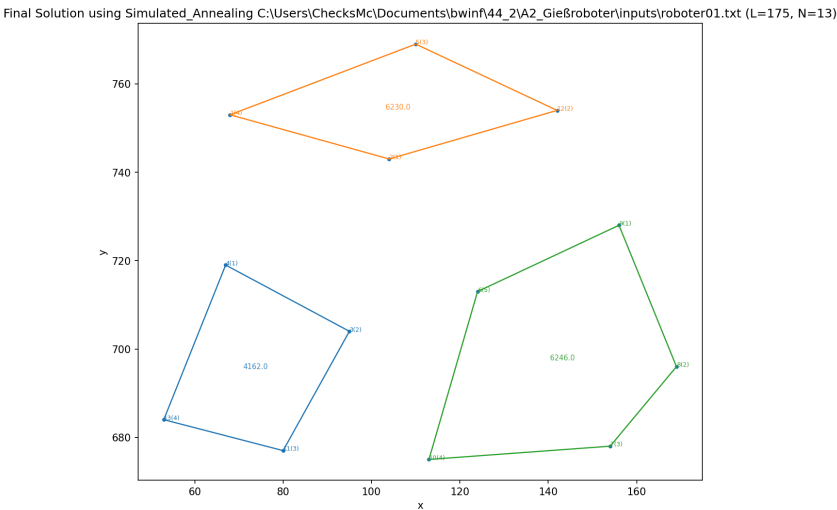
Simulierte Abkühlung

Ergebnis:

3
175
129
67 719
4 3 11 13 4
158
104 743
2 12 5 1 2
174
156 728
9 8 7 10 6 9

Laufzeit: 3.1687 s

Plot:



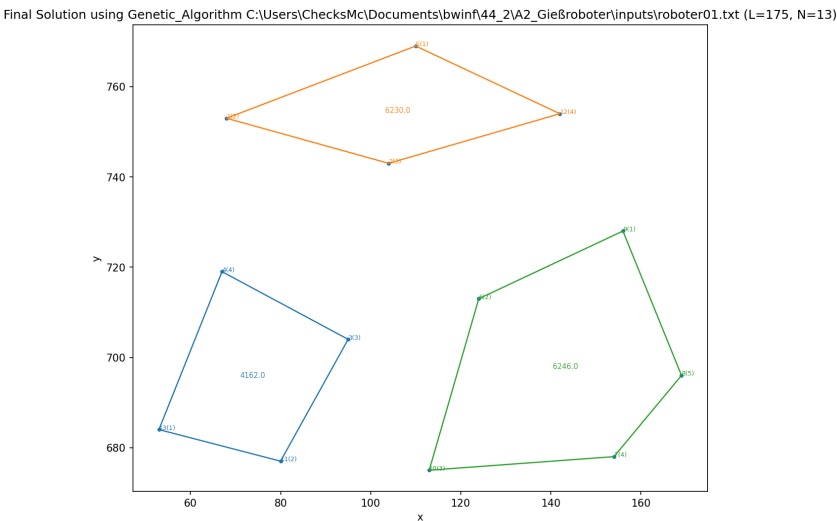
Genetischer Algorithmus

Ergebnis:

3
175
129
53 684
13 11 3 4 13
158
110 769
5 1 2 12 5
174
156 728
9 6 10 7 8 9

Laufzeit: 0.3124 s

Plot:



roboter02.txt

Initiallösung

Ergebnis:

9
175

44. Bundeswettbewerb Informatik - 2. Runde - Aufgabe A2: Gießroboter Teilnahme-ID: 79280

0
49 607
14
0
171 607
15
125
169 696
8 7 10 8
151
53 684
13 11 4 3 13
126
124 713
6 9 2 6
155
68 753
1 5 12 1
0
472 789
18
0
393 712
17
0
291 638
16

Laufzeit: 0.0000 s

Simulierte Abkühlung

Ergebnis:

8
175
0
171 607
15
129
95 704
3 11 13 4 3
174
156 728
9 8 7 10 6 9
0
393 712
17
0
291 638
16
158
68 753
1 2 12 5 1
0
49 607
14
0
472 789
18

Laufzeit: 2.8078 s

Genetischer Algorithmus

44. Bundeswettbewerb Informatik - 2. Runde - Aufgabe A2: Gießroboter Teilnahme-ID: 79280

Ergebnis:

```
9
175
0
49 607
14
61
124 713
6 3 6
0
171 607
15
163
67 719
4 13 11 10 4
109
154 678
7 8 9 7
158
104 743
2 1 5 12 2
0
472 789
18
0
393 712
17
0
291 638
16
```

Laufzeit: 0.3240 s

roboter03.txt

Initiallösung

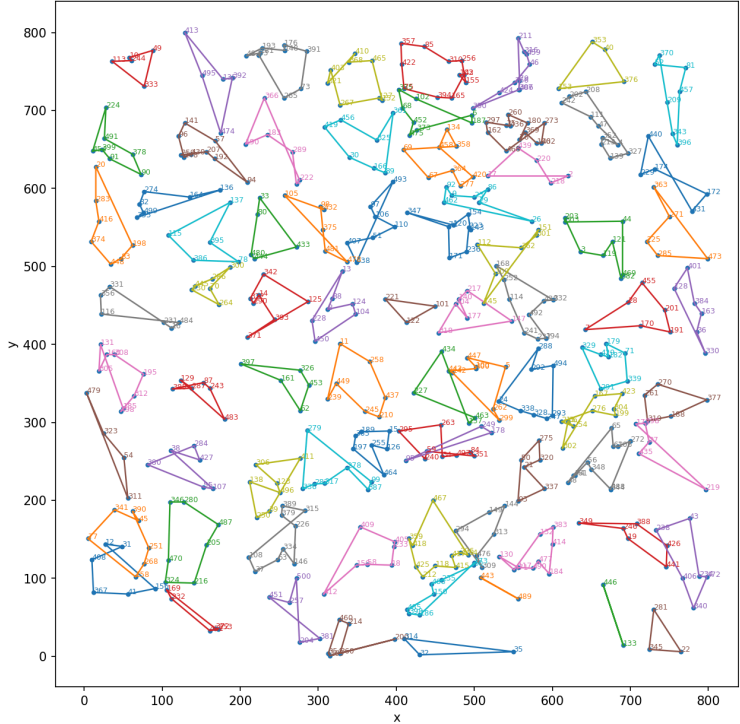
Ergebnis:

```
76
300
```

Laufzeit: 0.0000 s

Plot:

Initial Solution C:\Users\ChecksMc\Documents\bwinf44_2\A2_Gießroboter\inputs\roboter03.txt (L=300, N=500)



Simulierte Abkühlung

Ergebnis:

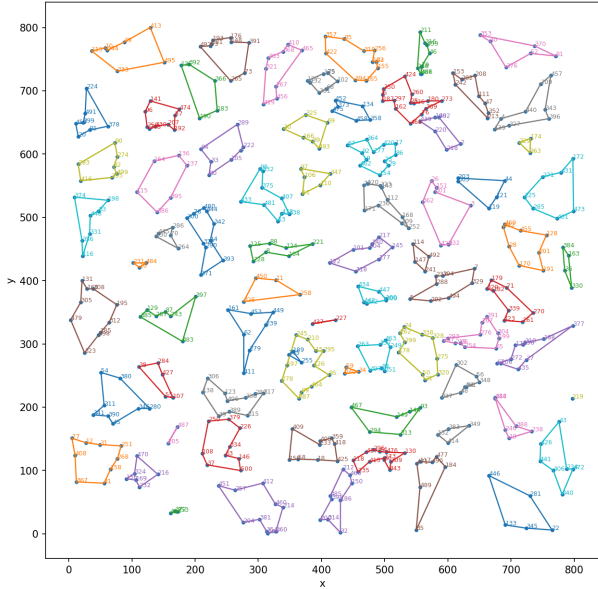
71

300

Laufzeit: 134.0869 s

Plot:

Final Solution using Simulated_Annealing C:\Users\ChecksMc\Documents\bwinf44_2\A2_Gießroboter\inputs\roboter03.txt (L=300, N=500)



Genetischer Algorithmus

Ergebnis:

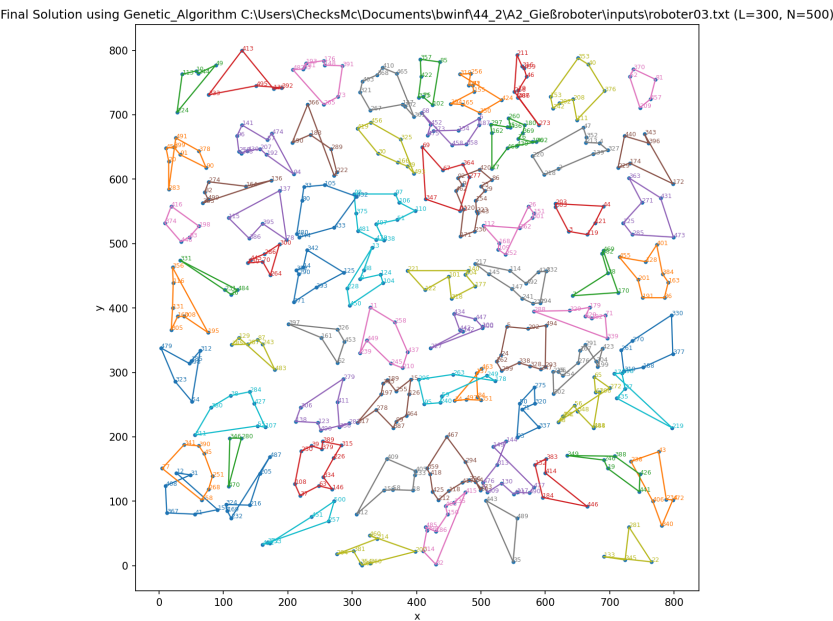
69

300

44. Bundeswettbewerb Informatik - 2. Runde - Aufgabe A2: GießroboterTeilnahme-ID: 79280

Laufzeit: 22.1054 s

Plot:



roboter04.txt

Initiallösung

Ergebnis:

47
100

Laufzeit: 0.0000 s

Simulierte Abkühlung

Ergebnis:

41
100

Laufzeit: 48.2514 s

Genetischer Algorithmus

Ergebnis:

44
100

Laufzeit: 3.4349 s

roboter05.txt

Initiallösung

Ergebnis:

4
1400
1320
379 224
121 129 103 111 107 114 124 132 117 127 130 105 128 116 126 131 108 112 113 122 125 102 106
110 115 119 118 133 123 104 77 101 73 83 86 95 100 74 89 99 82 70 81 71 90 84 87 78 97 92 94
98 76 72 88 69 96 93 121
1309

44. Bundeswettbewerb Informatik - 2. Runde - Aufgabe A2: GießroboterTeilnahme-ID: 79280

208 399

75 80 85 91 79 185 184 196 190 176 183 197 193 170 191 182 181 180 174 189 177 178 171 199 169
187 168 186 172 192 200 198 175 179 194 188 5 11 16 173 29 12 19 9 27 24 15 20 14 34 31 17 25
6 22 26 1 3 10 8 33 13 28 18 23 2 30 75

1132

242 708

7 21 32 4 195 157 152 138 150 162 146 144 153 135 156 158 167 141 159 165 143 166 154 164 147
136 155 161 142 163 137 139 151 149 160 148 145 140 7

1254

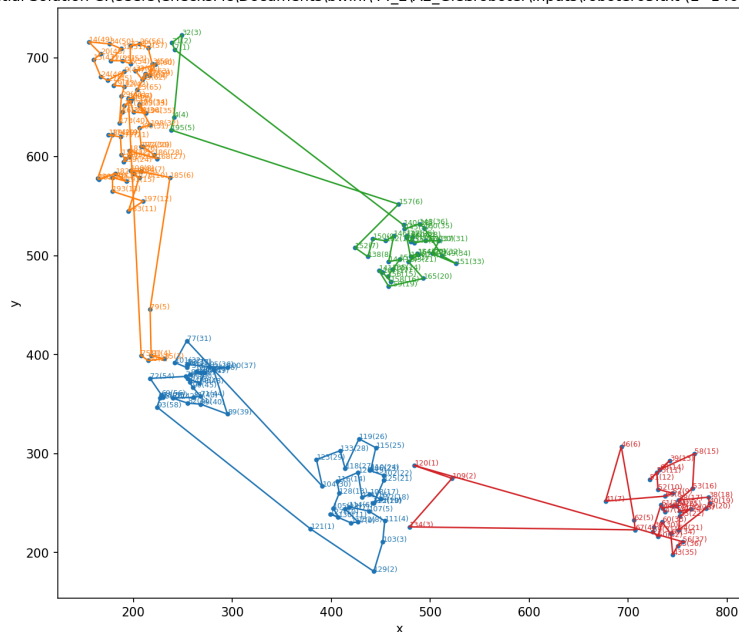
484 288

120 109 134 67 62 46 41 66 57 52 55 51 39 68 58 53 48 38 40 49 54 35 42 37 64 45 44 47 61 36
59 50 60 65 43 63 56 120

Laufzeit: 0.0009 s

Plot:

Initial Solution C:\Users\ChecksMc\Documents\bwinf44_2\A2_Gießroboter\inputs\roboter05.txt (L=1400, N=200)



Simulierte Abkühlung

Ergebnis:

4

1400

1048

439 283

106 110 102 115 119 133 100 95 74 97 94 92 86 73 76 87 84 90 81 78 98 83 77 79 75 80 91 101 85
72 69 88 96 93 70 82 99 71 89 123 104 128 116 105 127 121 130 124 114 122 113 112 108 118 126
106

704

209 678

13 10 28 8 3 21 32 7 1 26 22 31 27 19 29 30 11 16 173 189 174 177 187 199 169 192 172 196 191
182 181 180 170 193 183 197 176 185 168 186 184 190 171 178 194 175 5 2 12 13

1213

476 520

147 145 140 157 4 18 33 6 17 20 24 15 14 34 25 9 23 179 188 200 198 195 152 138 150 162 146
155 142 136 161 148 160 163 137 151 149 139 165 154 143 144 167 158 159 156 141 135 153 164
166 147

1115

733 248

61 44 47 35 37 45 60 65 54 43 63 56 50 59 36 67 62 41 134 111 103 129 117 132 107 131 125 120
109 46 51 55 68 39 58 53 38 40 49 42 64 48 57 66 52 61

44. Bundeswettbewerb Informatik - 2. Runde - Aufgabe A2: GießroboterTeilnahme-ID: 79280

Laufzeit: 79.3467 s

Plot:



Genetischer Algorithmus

Ergebnis:

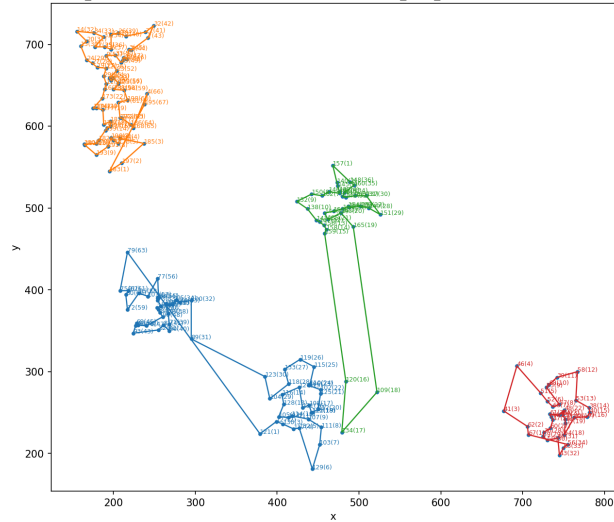
4
1400
1287
379 224
121 127 130 117 132 129 103 111 107 114 124 105 128 116 126 131 108 113 122 112 125 102 106
110 115 119 133 118 104 123 89 100 74 95 97 94 92 78 71 99 81 82 93 96 69 88 70 90 84 87 76 98
73 86 83 77 101 85 72 80 91 75 79 121
891
195 545
183 197 185 184 176 190 196 191 193 181 180 170 182 199 171 169 187 178 177 174 189 173 16 11
29 12 19 27 24 15 20 14 34 31 17 25 6 22 26 1 21 32 7 8 3 10 28 18 13 33 9 23 30 5 2 179 175
188 194 198 200 192 172 186 168 4 195 183
897
468 552
157 140 145 147 136 146 162 150 152 138 141 167 156 158 159 120 134 109 165 143 135 144 153
164 154 166 139 149 151 137 163 142 155 161 160 148 157
546
707 223
67 62 41 46 51 52 66 57 55 68 39 58 53 38 40 49 42 54 35 37 64 48 45 61 44 47 60 36 59 50 65
43 63 56 67

Laufzeit: 34.6160 s

Plot:

44. Bundeswettbewerb Informatik - 2. Runde - Aufgabe A2: Gießroboter Teilnahme-ID: 79280

Final Solution using Genetic_Algorithm C:\Users\ChecksMc\Documents\bwinf44_2\A2_Gießroboter\inputs\roboter05.txt (L=1400, N=200)



roboter06.txt

Initiallösung

Ergebnis:

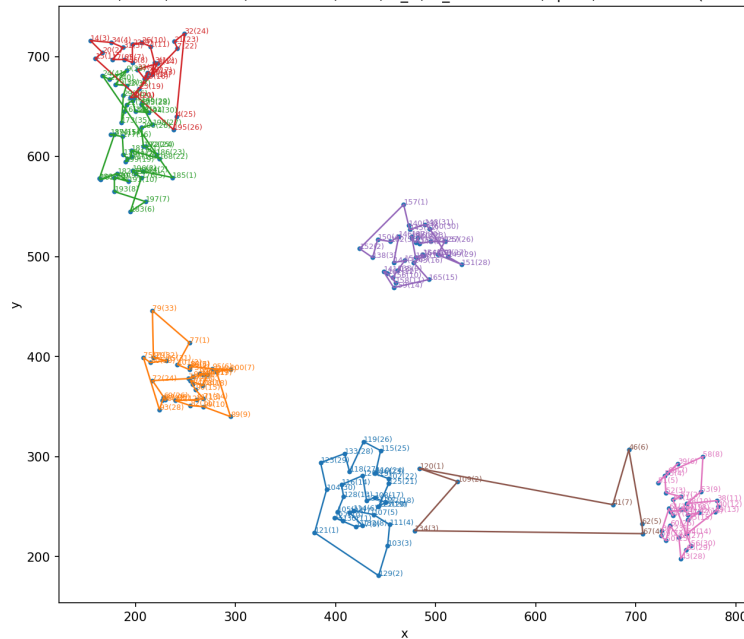
```
7
700
586
379 224
121 129 103 111 107 114 124 132 117 127 130 105 128 116 126 131 108 112 113 122 125 102 106
110 115 119 118 133 123 104 121
569
254 414
77 101 73 83 86 95 100 74 89 99 82 70 81 71 90 84 87 78 97 92 94 98 76 72 88 69 96 93 75 80 85
91 79 77
687
237 579
185 184 196 190 176 183 197 193 170 191 182 181 180 174 189 177 178 171 199 169 187 168 186
172 192 200 198 175 179 194 188 5 11 16 173 29 12 19 9 27 24 185
538
160 698
15 20 14 34 31 17 25 6 22 26 1 3 10 8 33 13 28 18 23 2 30 7 21 32 4 195 15
503
468 552
157 152 138 150 162 146 144 153 135 156 158 167 141 159 165 143 166 154 164 147 136 155 161
142 163 137 139 151 149 160 148 145 140 157
673
484 288
120 109 134 67 62 46 41 120
486
737 257
66 57 52 55 51 39 68 58 53 48 38 40 49 54 35 42 37 64 45 44 47 61 36 59 50 60 65 43 63 56 66
```

Laufzeit: 0.0000 s

Plot:

44. Bundeswettbewerb Informatik - 2. Runde - Aufgabe A2: GießroboterTeilnahme-ID: 79280

Initial Solution C:\Users\ChecksMc\Documents\bwinf\44_2\A2_Gießroboter\inputs\roboter06.txt (L=700, N=200)



Simulierte Abkühlung

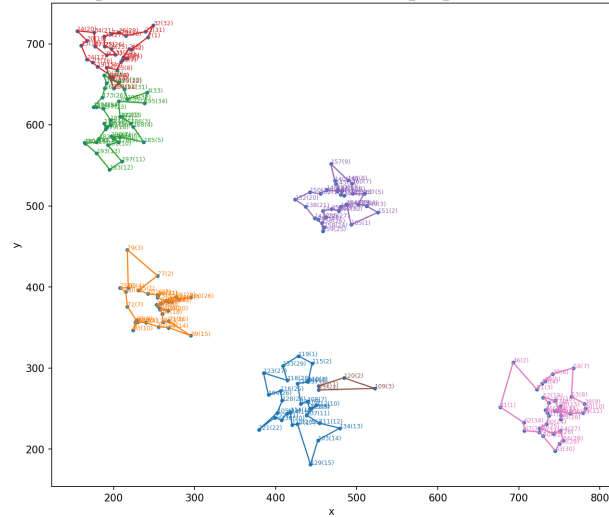
Ergebnis:

```
7
700
531
428 315
119 115 110 106 126 131 108 113 122 112 107 111 134 103 129 132 117 114 124 130 127 121 105
128 116 104 123 118 133 119
471
231 396
85 77 79 91 75 80 72 69 96 93 88 70 82 99 89 71 81 90 84 78 87 76 98 94 97 100 74 95 92 73 83
86 101 85
465
208 610
192 172 186 168 185 184 196 190 176 191 197 183 193 180 181 170 182 199 178 171 169 187 177
174 189 173 16 11 29 179 194 198 4 195 200 192
390
242 708
7 8 3 28 10 18 13 23 12 30 188 175 5 2 19 27 24 15 20 14 34 17 9 33 6 25 31 22 26 1 21 32 7
399
493 477
165 151 149 139 137 163 160 148 157 140 145 142 155 161 147 136 146 162 150 152 138 141 167
158 159 156 135 144 153 143 164 154 166 165
147
453 278
102 120 109 125 102
523
677 252
41 46 51 55 68 39 58 53 38 40 49 42 64 45 48 35 37 57 52 66 44 61 47 60 36 65 54 56 63 43 50
59 67 62 41
```

Laufzeit: 65.5552 s

Plot:

Final Solution using Simulated_Annealing C:\Users\ChecksMc\Documents\bwinf\44_2\A2_Gießroboter\inputs\roboter06.txt (L=700, N=200)



Genetischer Algorithmus

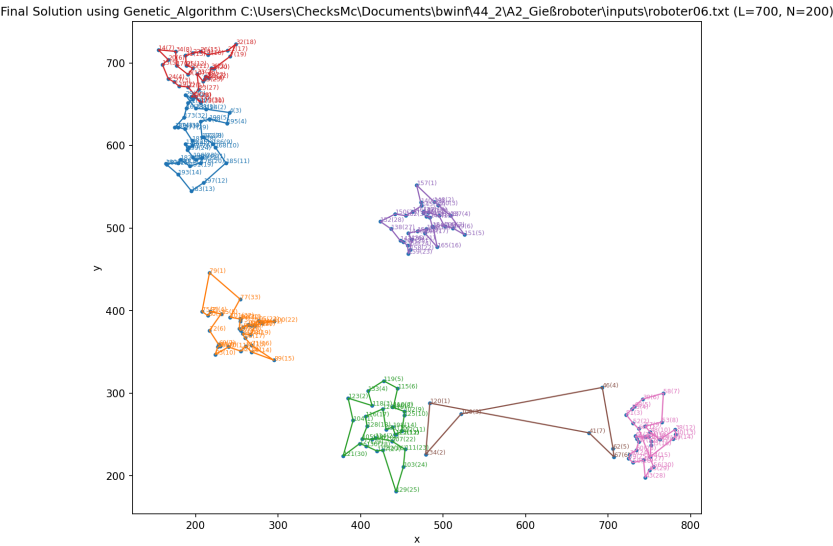
Ergebnis:

```

7
700
453
200 645
188 194 4 195 198 200 192 172 186 168 185 197 183 193 181 180 170 182 191 176 184 190 196 199
178 171 169 187 177 174 189 173 16 11 5 29 188
461
217 446
79 75 80 91 85 72 69 88 96 93 70 82 81 99 89 71 90 84 78 97 74 100 95 94 92 98 87 76 73 83 86
101 77 79
546
391 267
104 123 118 133 119 115 110 106 102 125 112 122 113 108 131 126 116 128 105 124 114 107 111
103 129 132 117 130 127 121 104
374
191 671
12 19 27 24 15 20 14 34 17 9 6 25 31 22 26 1 21 32 7 8 3 10 28 18 13 33 23 30 2 175 179 12
407
468 552
157 148 160 137 151 149 139 163 161 147 136 155 142 154 166 165 143 164 153 144 135 158 159
156 167 141 138 152 150 162 146 145 140 157
625
484 288
120 134 109 46 62 67 41 120
407
737 257
66 52 51 55 68 39 58 53 57 48 42 38 40 49 54 35 37 64 45 61 44 47 60 36 59 50 65 43 63 56 66
    
```

Laufzeit: 7.8281 s

Plot:



roboter07.txt

Initiallösung

Ergebnis:

220
300

Laufzeit: 0.2008 s

Simulierte Abkühlung

Ergebnis:

220
300

Laufzeit: 1926.1454 s

Genetischer Algorithmus

Ergebnis:

205
300

Laufzeit: 4351.7577 s

roboter08.txt

Initiallösung

Ergebnis:

706
100

Laufzeit: 0.1626 s

Simulierte Abkühlung

Ergebnis:

704
100

Laufzeit: 1707.3017 s

Genetischer Algorithmus

44. Bundeswettbewerb Informatik - 2. Runde - Aufgabe A2: GießroboterTeilnahme-ID: 79280

Ergebnis:

690
100

Laufzeit: 2102.2852 s

roboter09.txt

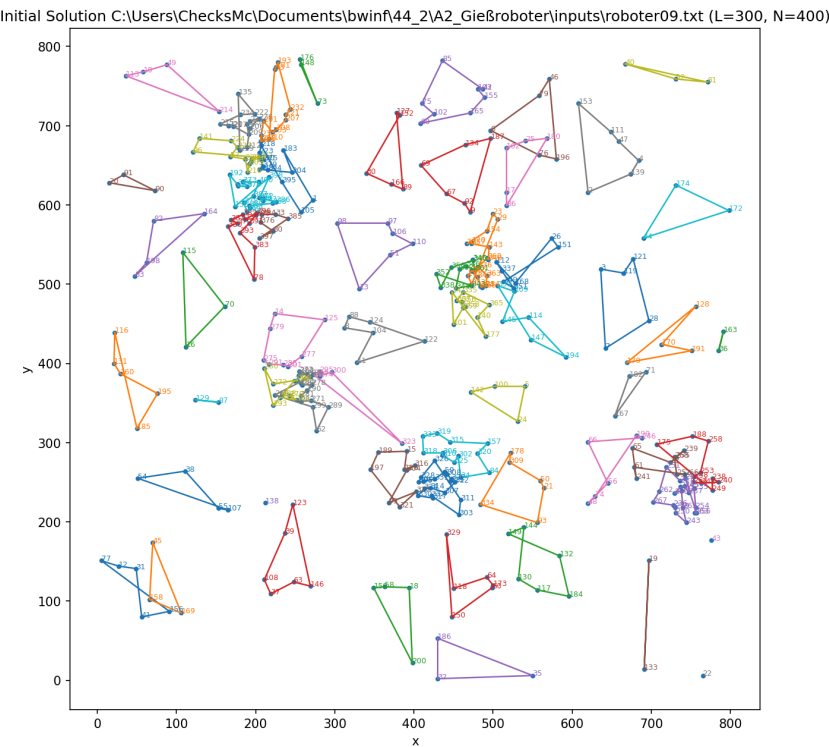
Initiallösung

Ergebnis:

58
300

Laufzeit: 0.0210 s

Plot:



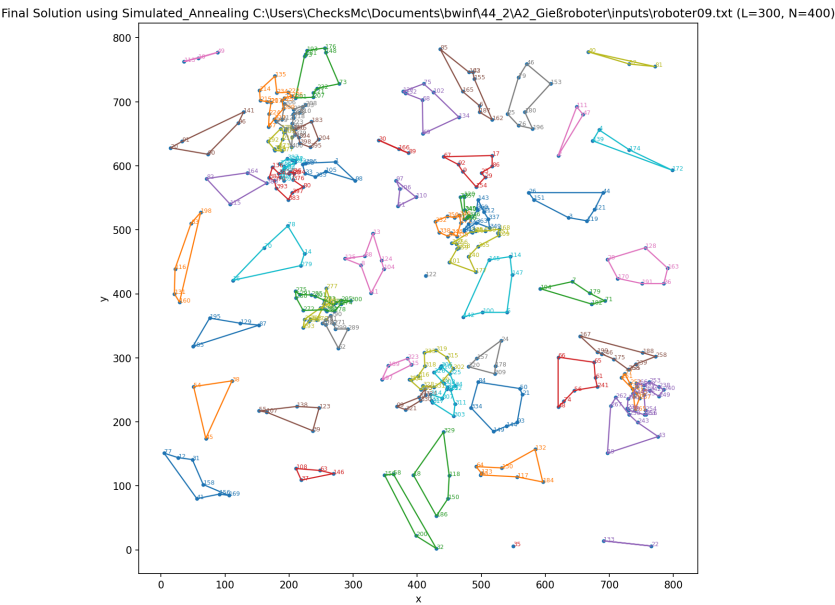
Simulierte Abkühlung

Ergebnis:

56
300

Laufzeit: 104.4882 s

Plot:



Genetischer Algorithmus

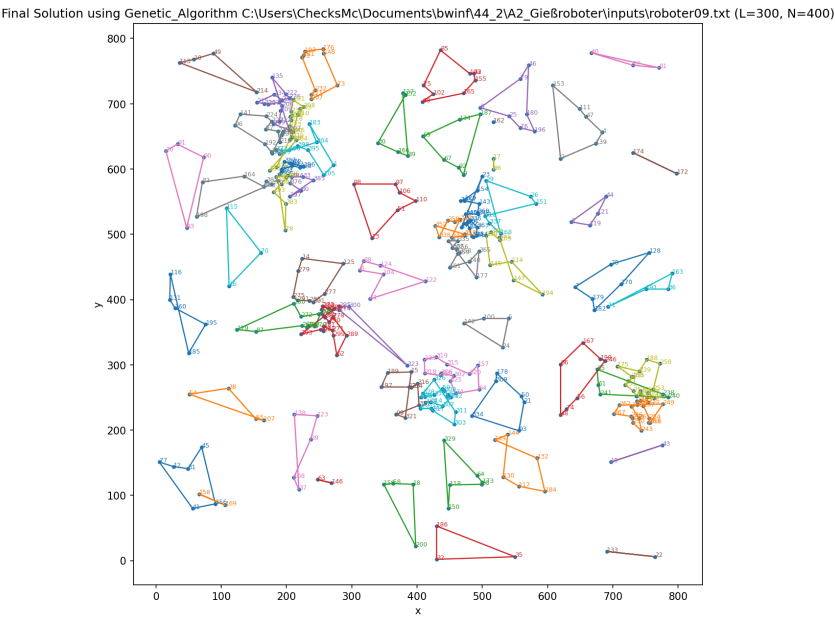
Ergebnis:

56

300

Laufzeit: 16.9551 s

Plot:



roboter10.txt

Initiallösung

Ergebnis:

593

300

Laufzeit: 0.1763 s

Simulierte Abkühlung

44. Bundeswettbewerb Informatik - 2. Runde - Aufgabe A2: Gießroboter Teilnahme-ID: 79280

Ergebnis:

591
300

Laufzeit: 1623.5589 s

Genetischer Algorithmus

Ergebnis:

585
300

Laufzeit: 2296.4548 s

roboter11.txt

Initiallösung

Ergebnis:

2395
300

Laufzeit: 2.2088 s

Simulierte Abkühlung

Ergebnis:

2394
300

Laufzeit: 7650.4929 s

Genetischer Algorithmus

Ergebnis:

2400
300

Laufzeit: 10287.5559 s

roboter12.txt

Initiallösung

Ergebnis:

2
10

Laufzeit: 0.0082 s

Simulierte Abkühlung

Ergebnis:

2
10

Laufzeit: 0.4588 s

Genetischer Algorithmus

Ergebnis:

2
10

Laufzeit: 0.3283 s

roboter13.txt

Initiallösung

Ergebnis:

1
1000

Laufzeit: 0.0059 s

Simulierte Abkühlung

Ergebnis:

1
1000

Laufzeit: 0.4306 s

Genetischer Algorithmus

Ergebnis:

1
1000

Laufzeit: 0.0096 s

roboter14.txt Der Fall mit 0 Bäumen hat den Failsafe in initialize.py ausgelöst, wodurch das Programm frühzeitig mit einer Fehlermeldung terminiert wurde. Das ist das erwartete Verhalten, da es keine gültige Lösung gibt.

10 Vergleich der Lösungsansätze

10.1 Einleitung zum Vergleich

In diesem Abschnitt werden die drei implementierten Lösungsansätze, also die deterministische Initiallösung via Hilbert-Kurve und Greedy-Algorithmus sowie die zwei metaheuristischen Optimierungsverfahren Simulated Annealing (SA) und Genetischer Algorithmus (GA). Anhand der im Beispiel Teil erzielten Ergebnisse werden die Verfahren verglichen. Betrachtet werden sowohl die Lösungsqualität (Also die Anzahl der Roboter) als auch die Laufzeit und das Verhalten auf unterschiedlich großen Instanzen.

Die Auswertung stützt sich auf die Beispiele roboter01-11 sowie auf die ergänzenden Randfälle roboter12-13. Dadurch wird sichtbar, dass die Verfahren nicht auf jeder Instanz gleich reagieren: Auf kleinen Dateien ist der Abstand zwischen den Verfahren gering, während bei größeren und schwieriger strukturierten Instanzen deutliche Unterschiede auftreten.

10.2 Vergleich nach Effizienz

Die Laufzeitgrafik zeigt zunächst einen klaren Unterschied zwischen der Initiallösung und den beiden Optimierern: Hilbert-Greedy liefert praktisch sofort eine gültige Startlösung, während SA und GA den Suchraum aktiv weiter durchsuchen und dafür deutlich mehr Rechenzeit benötigen.

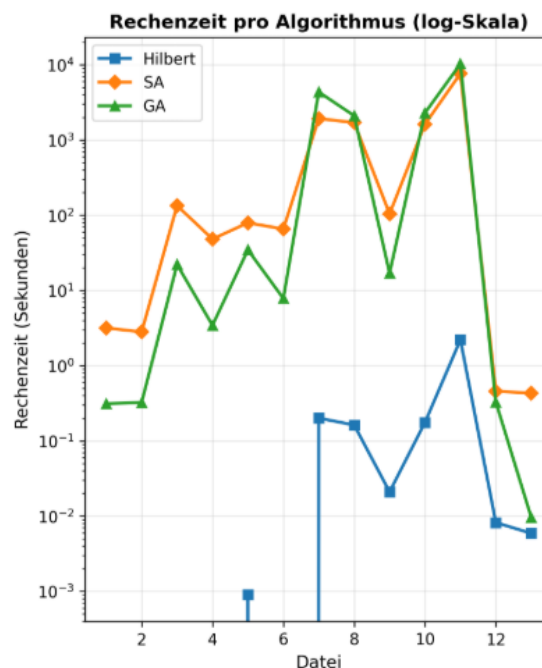


Abbildung 10: Laufzeitvergleich der drei Ansätze.

Für die mittleren und großen Instanzen ist vor allem wichtig, dass die Laufzeiten nicht linear verlaufen, sondern stark von der Problemgröße und von der jeweiligen Struktur der Eingabedatei abhängen. Auf den kleineren Beispielen sind beide Metaheuristiken noch vergleichsweise schnell, bei den größeren Instanzen steigen die Laufzeiten jedoch sprunghaft an. Was auch in der logarithmisch skalierten Grafik zu sehen ist.

Es fällt auf, dass GA und SA ähnliche Laufzeiten haben, wobei GA tendenziell etwas schneller ist, was die Wahl von GA als Hauptoptimierer unterstreicht. Die Grafik verdeutlicht, dass beide Verfahren ein ähnliches Wachstum zeigen, GA aber bei manchen Fällen stabiler ist.

Für die praktische Nutzung bedeutet das, dass die Initiallösung die schnellste und verlässlichste Option ist, während GA einen guten Kompromiss aus Laufzeit und Verbesserung, und SA ist die rechenintensivere Variante mit dem Potenzial für noch bessere Endlösungen auf schwierigen Instanzen.

Es ist anzumerken, dass die Laufzeit durch Implementierungsineffizienzen, die Parameterwahl und die Programmiersprache Python beeinflusst wird. In unvollständigen, aus Zeitgründen nicht final integrierten Tests zeigten

Erweiterungen (z. B. Numba) sowie eine gezieltere Gewichtungswahl das Potenzial, die heuristische Suche stärker zu führen und die Laufzeit deutlich zu reduzieren.

10.3 Leistung

Bei der Qualität fällt auf, dass die Initiallösung im Vergleich zu den teilweise potenziell suboptimalen Optimierer-Ergebnissen relativ stabil ist. Die Metaheuristiken verbessern in den meisten Fällen die Startlösung, die Verbesserungen sind jedoch nicht immer stark ausgeprägt. Dies wird auch in der Grafik deutlich, in der sich die drei Kurven teilweise überlappen.

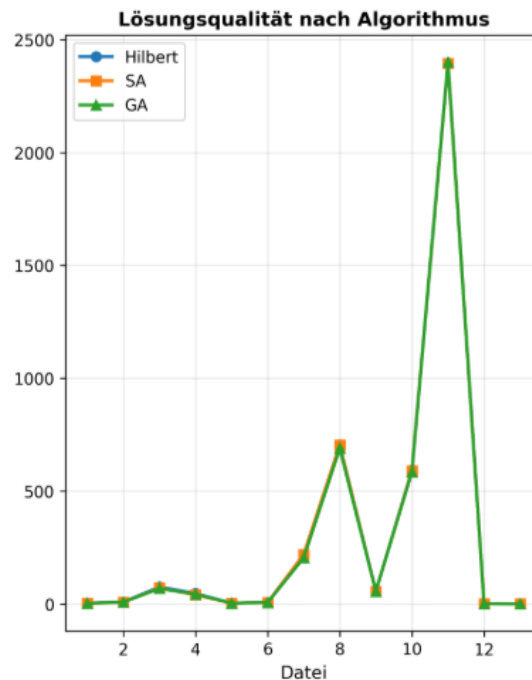


Abbildung 11: Lösungsqualität nach Algorithmus.

Die Auswertung zeigt, dass GA nicht in allen Fällen überlegen ist. Bei manchen vielen Instanzen ist SA gleich gut oder sogar etwas besser, und bei den Randfällen roboter12 und roboter13 bringen beide Optimierer keinen zusätzlichen Gewinn, weil die Startlösung bereits optimal beziehungsweise praktisch optimal ist.

Zusammenfassend lässt sich die Qualitätsbewertung so lesen, dass GA im Mittel die etwas bessere Endlösung liefert, SA dafür häufiger nah an der besten hier gefundenen Lösung liegt und in den schwierigen Fällen nicht weit entfernt ist. Es fällt jedoch auch auf, dass GA zum Beispiel mit dem aktuellen Seed und den Parametern bei roboter11 eine deutlich schlechtere Lösung findet als Hilbert, was stark darauf hindeutet, dass die Parametrierung suboptimal ist.

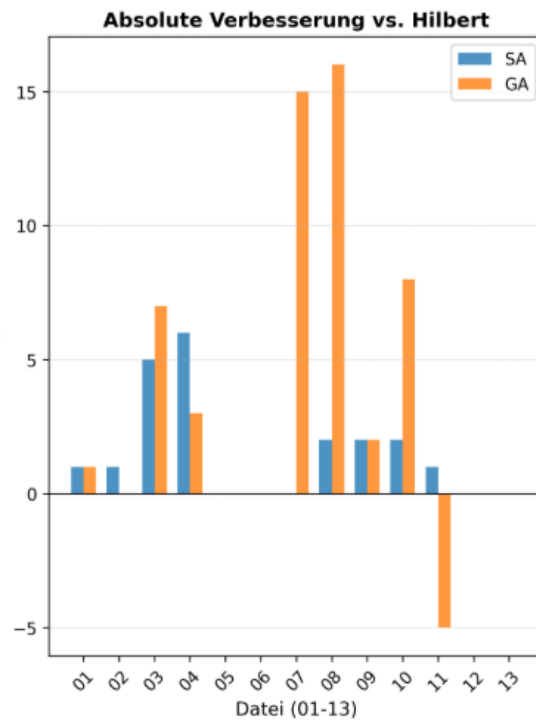


Abbildung 12: Absolute Verbesserung gegenüber der Hilbert-Baseline.

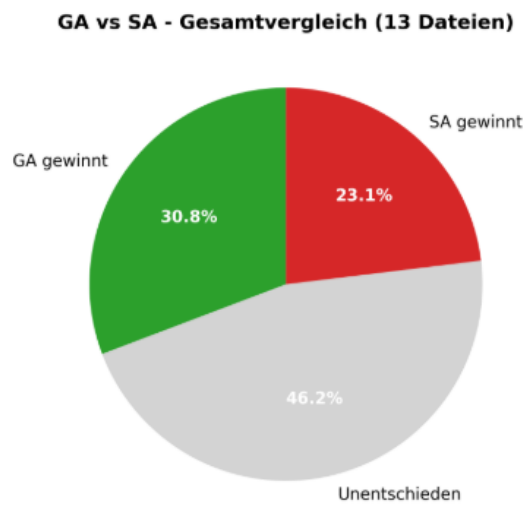


Abbildung 13: Direktvergleich GA gegen SA.

Die Verteilung zeigt, dass GA nicht in allen Fällen besser ist, aber in mehr Fällen eine stärkere Verbesserung als SA erzielt.

11 Fazit & Ausblick

Die Kombination aus Hilbert-Kurve und Greedy-Initialisierung bleibt die stärkste Basis des gesamten Ansatzes. Sie liefert in sehr kurzer Zeit eine valide Startlösung und bildet damit die notwendige Grundlage für die anschließende Optimierung.

Die Randfälle zeigen außerdem, dass die Implementierung korrekt mit bereits optimalen oder fast optimalen Startlösungen umgehen kann. Dort ist keine weitere Verbesserung nötig, und genau das ist fachlich erwünscht.

Es fällt auf, dass die Hilbert-Greedy-Initialisierung bereits eine sehr gute Lösung liefert, weshalb die teilweise nicht optimalen Lösungen der Optimierer weniger stark ins Gewicht fallen. Dadurch erzielen die Optimierer in den meisten Fällen nur geringe Verbesserungen gegenüber der Hilbert-Baseline und schneiden in manchen Fällen sogar schlechter ab.

Weitere Erweiterungen könnten an folgenden Punkten ansetzen:

- feinere Parametrisierung von SA und GA auf Basis separater Instanzklassen und eine tiefere und systematischere Parametrierung, um die Leistung der Optimierer zu verbessern,
- Laufzeitoptimierungen durch effizientere Datenstrukturen, die Nutzung von Numba oder die Implementierung von Kernkomponenten in einer schnelleren Sprache wie C++,
- optional ein deterministisches Referenzverfahren für kleine Instanzen, um die Heuristiken noch präziser einzuordnen.

12 Vergleich mit KI-Numba Ansatz

Dieser Abschnitt vergleicht die Basisimplementierung (solver) mit dem KI-unterstützt erzeugten Numba-Variantenstand (solver_numba). Wichtig ist dabei: Der Numba-Ansatz verändert nicht nur die Ausführungsgeschwindigkeit einzelner Schleifen, sondern an mehreren Stellen auch das Optimierungsverhalten von SA und GA.

12.1 Konkrete Implementierungsunterschiede

1) SA: Nachbarschaftssuche und Bewertung

In der Basisversion erzeugt SA Nachbarn überwiegend über zufällige Relocate-, Swap-, Shuffle- und 2-Opt-Schritte mit temperaturabhängiger Auswahl. In der Numba-Variante werden diese Schritte durch zusätzliche Kernel-Funktionen ergänzt, die gezieltere, feasibility-nahe Moves bevorzugen (u. a. best-position-relocate, best-pair-swap und explizite Merge-Versuche).

Vereinfacht ausgedrückt:

- Basis-SA: „zufälliger Move, dann akzeptieren oder verwerfen“
- Numba-SA: „strukturierter Move-Kandidat, dann akzeptieren oder verwerfen“

Zusätzlich nutzt die Numba-Variante eine strengere Zielführung über dynamische Penalty-Ramping-Strategien und Grenzprüfungen für kritische Überlängen. Das reduziert häufig die Zahl klar unbrauchbarer Zwischenzustände, kann aber die Exploration des Suchraums auch stärker einschränken.

2) GA: Fitness, Crossover und lokale Suche

In der Basisversion verwendet GA eine klassische OX1-Rekombination, Inversionsmutation und optionale lokale Verbesserung (2-Opt) pro Generation. In der Numba-Variante werden Fitness-Metriken über Kernel-Funktionen berechnet, die Strafterme dynamischer skaliert und zusätzliche Mutations-/Relokationsideen eingebaut.

Außerdem wurde das Crossover-Verhalten verändert: Statt reinem OX1 steht stärker die Übernahme bereits guter Routenstrukturen im Vordergrund. Das kann pro Generation schnellere Qualitätsanstiege liefern, erhöht aber gleichzeitig das Risiko sinkender Populationsdiversität, wenn die Strafgewichte zu dominant sind.

3) Konfigurationsänderungen

Neben den Codepfaden wurden auch Parameter angepasst, insbesondere im Large-Bereich (u. a. andere Alpha/Beta/Gamma-Gewichtungen, Population/Generationen und Margen). Damit ist der Vergleich kein reiner „Compiler-Speedup“, sondern ein Vergleich zweier unterschiedlich gesteuerter Metaheuristik-Varianten.

12.2 Beispiellun roboter03 (SA vs. GA)

Zur Einordnung ein konkretes Beispiel auf roboter03.txt in der Basisimplementierung:

Numba-Run:

Algorithmus 2 (SA):

Final solution has 58 routes

Solver runtime: 161.8398 s

Algorithmus 3 (GA):

Final solution has 73 routes

Solver runtime: 3.0686 s

Bisherige Werte:

SA:

71 Roboter

134.0869 s

GA:

69 Roboter

22.1054 s

Vergleich:

SA: Qualität besser (58 statt 71 Roboter) Laufzeit schlechter (161.84 s statt 134.09 s) GA: Laufzeit viel besser (3.07 s statt 22.11 s) Qualität schlechter (73 statt 69 Roboter)

Beide Verfahren zeigen also eine signifikante Veränderung gegenüber der Basisversion, aber in entgegengesetzte Richtungen: SA liefert eine deutlich bessere Lösung, benötigt dafür aber etwas mehr Zeit, während GA deutlich schneller ist, aber in diesem Fall eine schlechtere Lösung liefert.

12.3 Einordnung des KI-Numba-Ansatzes

Der KI-Numba-Stand ist als Engineering-Variante mit zwei Zielen zu verstehen: (1) reduzierte Rechenkosten in Hotspots und (2) stärkere Zielführung auf das Primärziel „Roboteranzahl minimieren“. Der Preis dafür ist ein methodischer Trade-off: Je stärker Penalties, Filter und Gewichtungen angesteuert werden, desto robuster ist die Konvergenz in Richtung valider Lösungen, aber desto höher kann das Risiko einer zu frühen Verengung des Suchraums sein.

Deshalb bleibt für eine faire Bewertung entscheidend, beide Varianten unter identischen Bedingungen zu vergleichen (gleiche Instanzen, gleiche Seeds, gleiche Restart-Regeln, gleiche Validierung). Erst dann lässt sich beurteilen, ob die Numba-Variante in einer Instanzklasse tatsächlich einen besseren Gesamtkompromiss aus Laufzeit, Roboteranzahl und Stabilität liefert.

13 Literaturverzeichnis

Bibliografie

- [1] D. D. Zoran Injac, „Classification of vehicle routing problem“. [Online]. Verfügbar unter: https://www.researchgate.net/publication/382151758_CLASSIFICATION_OF_VEHICLE_ROUTING_PROBLEM
- [2] C. B.-S. 4.0 Parmo, „File:Hilbert curve inverse.svg - Wikimedia Commons“. Zugegriffen: 22. Mai 2024. [Online]. Verfügbar unter: <https://commons.wikimedia.org/w/index.php?curid=64870340>
- [3] Braindrain, „Datei:Hilbert curve!.gif“. Zugegriffen: 22. Mai 2024. [Online]. Verfügbar unter: https://de.wikipedia.org/wiki/Datei:Hilbert_curve!.gif
- [4] Wikipedia contributors, „Genetic Algorithm“. [Online]. Verfügbar unter: https://en.wikipedia.org/wiki/Genetic_algorithm
- [5] N. A. Wouda, L. Lan, und W. Kool, „PyVRP: a high-performance VRP solver package“. [Online]. Verfügbar unter: <https://arxiv.org/abs/2403.13795>
- [6] L. Alseda, „Genetic Operations“. [Online]. Verfügbar unter: <https://web.archive.org/web/20251220164819/https://mat.uab.cat/~alseda/MasterOpt/GeneticOperations.pdf>
- [7] Wikipedia Contributors, „Simulated Annealing“. [Online]. Verfügbar unter: https://en.wikipedia.org/wiki/Simulated_annealing
- [8] G. Iyer, „Simulated Annealing (Brief Lecture Notes)“. [Online]. Verfügbar unter: <https://www.math.cmu.edu/~gautam/c/2024-387/notes/10-simulated-annealing.html>

14 Quellcode

Dieses Kapitel zeigt die Implementierung direkt über Struktur und zentrale Codeausschnitte.

14.1 Hauptablauf

```
def main():
    trees, max_distance, node_ids, distmatrix = initialize(filepath)
    hilbert_list = hilbert_sort(trees, node_ids)
    initial_routes = greedy(hilbert_list, max_distance, distmatrix)

    valid, _ = validity_check(initial_routes, distmatrix, max_distance, node_ids)
    best_routes = initial_routes
    algo = "Hilbert"

    if algorithm == 2:
        best_routes = sa_improve(
            trees,
            [route[:] for route in initial_routes],
            max_distance,
            distmatrix,
            sa_restarts=args.sa_restarts,
        )
        algo = "Simulated_Annealing"
    elif algorithm == 3:
        best_routes = ga_improve(
            trees,
            [route[:] for route in initial_routes],
            max_distance,
            distmatrix,
            ga_restarts=args.ga_restarts,
        )
        algo = "Genetic_Algorithm"

    valid, _ = validity_check(best_routes, distmatrix, max_distance, node_ids)
    if (not valid) or len(best_routes) > len(initial_routes):
        best_routes = initial_routes

    log(..., destination=args.results_file)
    printout(best_routes, max_distance, filepath, distmatrix, algo, trees)
```

14.2 Einlesen + Distanzmatrix

```
def readfile(filepath):
    maxdist = int(first_line)
    node_count = int(node_count_line)
    # ...
    if int(tree_id) != expected_id:
        raise ValueError("Tree IDs must be sequential starting from 1")

def precompute_dist_matrix(ids, trees, dtype=np.float32):
    coords = np.array([trees[node_id] for node_id in ids], dtype=dtype)
    dx = coords[:, 0][:, None] - coords[:, 0][None, :]
    dy = coords[:, 1][:, None] - coords[:, 1][None, :]
    dist_matrix = dx * dx + dy * dy
    return dist_matrix
```

14.3 Initiallösung: Hilbert + Greedy

```
def hilbert_sort(trees, ids):
    # ...
    hilbert_sorted_nodes.sort(key=lambda x: x[1])
    return [node_pair[0] for node_pair in hilbert_sorted_nodes]

def greedy(hilbert, maxdist, distmatrix):
```

```

routes = []
current_route = []
current_length = 0.0

for next_node_id in hilbert:
    # ...
    if current_length + dist_to_next + dist_back_to_start <= maxdist:
        current_route.append(next_node_id)
    else:
        routes.append(current_route)
        current_route = [next_node_id]

```

14.4 Simulated Annealing (SA)

```

def energy(solution, maxdist, temperature, distmatrix):
    num_routes = len(solution)
    penalty_coeff = SAconfig.SA_TMAX / max(temperature, 1e-10)
    # ...
    return SA_ALPHA * num_routes + SA_BETA * over_length_penalty + SA_GAMMA * total_length

def neighbour(solution, temperature, init_temperature, rng=None):
    # relocate / swap / shuffle / 2-opt
    # ...

def sa_improve(coords, base, maxdist, distmatrix, test=False):
    while current_temp > min_temp:
        change = neighbour(...)
        # ...
        if energy_delta < 0 or random.random() < math.exp(-energy_delta / current_temp):
            # move akzeptieren
            # ...

```

14.5 Genetischer Algorithmus (GA)

```

def calculate_fitness(chromosome, maxlength, distmatrix, generation, generations):
    routes, total_dist, routelengths, overlengths = unflatten_and_score(...)
    penalty_coeff = 100 + 900 * (generation / max(1, generations))
    # ...

def ga_improve_single_run(trees, base_routes, maxlength, dist_matrix, seed):
    population = [...]
    for generation in range(generations):
        scored = [(c, calculate_fitness(...)) for c in population]
        # ...
        child = orderx(p1, p2)
        child = mutate(child)

```

14.6 Validierung + Ausgabe

```

def validity_check(routes, distmatrix, maxdist, node_ids):
    for route in routes:
        if route_length(route, sqrt_matrix) > maxdist:
            return False

    if len(all_visited_trees) != len(set(all_visited_trees)):
        return False

    if missing_trees:
        return False

    return True

def printout(routes, max_length, infile, squared_distmatrix, algo, tree, folder=None):
    f.write(f"{k}\n{max_length}\n")

```

```
for route in routes:  
    # ...
```

14.7 Ergänzende Module

- Visualisierung
- Logging
- Konfigurationsparameter